A Blockchain-Integrated Framework for Secure, Incentivized, and Scalable Machine Learning: Integrating Consensus Mechanisms and Reinforcement Learning

by

MD. ABRAR-UD-DOULA AMIYA
21301567
SHADMAN SAKIB
21301566
MAHIMA ISLAM
21201143
NAFI ALIF MAHIN
21201699
MUNTASIR MAHIN SIAM
21301708

A thesis submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
B.Sc. in Computer Science and Engineering

Department of Computer Science and Engineering
Brac University
June 2026.

Declaration

It is hereby declared that

1. The thesis submitted is my/our own original work while completing degree at Brac University.
2. The thesis does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
3. The thesis does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.
4. We have acknowledged all main sources of help.

Student's Full Name & Signature:

Md. Abrar-ud-doula Amiya

21301567

Shadman Sakib

21301566

Mahima Islam

21201143

Nafi Alif Mahin

21201699

Muntasir Mahin Siam

21301708

Approval

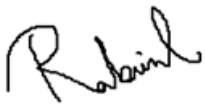
The thesis/project titled "A Blockchain-Integrated Framework for Secure, Incentivized, and Scalable Machine Learning: Integrating Consensus Mechanisms and Reinforcement Learning" submitted by

1. Md. Abrar-ud-doula Amiya (21301567)
2. Shadman Sakib (21301566)
3. Mahima Islam (21201143)
4. Nafi Alif Mahin (21201699)
5. Muntasir Mahin Siam (21301708)

of Spring, 2026 has been accepted as satisfactory in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science and Engineering on June 13, 2026.

Examining Committee:

Supervisor:
(Member)



Md. Golam Rabiul Alam, Ph.D.
Professor
CSE Department
Brac University

Thesis Coordinator:
(Member)

Md. Golam Rabiul Alam, PhD

Professor
Department of Computer Science and Engineering
Brac University

Head of Department:
(Chair)

Sadia Hamid Kazi, PhD

Chairperson and Associate Professor
Department of Computer Science and Engineering
Brac University

Abstract

The growth of digital financial services has made cross-institutional fraud faster to spread than any single bank can detect, yet privacy regulation and competitive distrust prevent institutions from pooling raw transaction data. This thesis presents FL-ADTCN, a blockchain-integrated federated learning framework for collaborative financial fraud detection that resolves the privacy, trust, and incentive barriers simultaneously. Each institution trains a local Adaptive Deep Temporal Context Network (ADTCN) – a temporal one-dimensional convolutional detector – on its own transactions, and only model parameters, never raw data, are shared. The federated aggregation layer combines *differentially private* weight sharing (Gaussian mechanism), *Krum* robust selection to reject outlying updates, and *Shapley-value* contribution attribution that quantifies each institution's marginal value to the global model; the Shapley weights are written to a Hyperledger Fabric ledger and drive an on-chain, chaincode-enforced token incentive. The central contribution is a **characterisation of the privacy–incentive coupling** of this mechanism: we show that differential privacy and contribution-based incentives are in direct tension, and that applying the privacy noise on the released contribution channel rather than on the shared weights restores rank-faithful, DP-protected on-chain rewards at roughly two orders of magnitude lower privacy budget ($\epsilon^\star$: $3000 \to 50$). This is supported by a characterisation suite that bounds the mechanism: the privacy–utility decoupling of accuracy versus reward fidelity; the economic isolation of a colluding majority; statistical Byzantine fault tolerance; and the cost and fidelity limits of Shapley attribution as the federation grows. The supporting substrate is a Dynamic Butterfly–Billiards Optimisation Algorithm (DB-BOA) that automates ADTCN hyperparameter tuning (without manual search, though it does not exceed a hand-tuned default) and consensus leader selection, together with a deliberately minimal reinforcement-learning (linear Q-learning) policy for *sequential* leader rotation that, in simulation, matches the optimiser on reward while adapting to nodes whose reliability degrades at runtime. The framework is implemented end-to-end on a real Hyperledger Fabric consortium with the `DBBOAContract` smart contract and evaluated on the public ULB Credit Card Fraud dataset (284,807 transactions, 0.17% fraud) partitioned across three banks. Detection metrics are reported at the deployment operating point, in which weight-channel differential privacy is disabled and privacy is instead enforced on the incentive channel, the configuration the contribution above shows to be both private and useful. *Scalability* is established for contribution *attribution* – a Monte-Carlo Shapley estimator that remains tractable to $n = 20$ organisations, faithful to $n \approx 5$ – rather than for blockchain throughput or distributed training, which remain in the limitations. *Security* is evaluated on two complementary fronts: Krum is run in the regime where its $n \geq 2f + 3$ guarantee holds ($n = 5, f = 1$ and $n = 7, f = 2$), rejecting four weight-level poisoning attacks, while the economic incentive layer isolates a behavioural colluding majority that out-numbers any statistical defence; the default three-organisation pipeline runs at $f = 0$ (consensus alignment, no adversary assumed).

Keywords: Federated Learning; FL-ADTCN; Adaptive Deep Temporal Context Network; Hyperledger Fabric; Permissioned Consortium Blockchain; Smart Con-

tracts; Financial Fraud Detection; Differential Privacy; Krum; Shapley Value; Contribution Attribution; Token Incentive Mechanism; Dynamic Butterfly–Billiards Optimisation (DB-BOA); Leader Selection; Privacy Preservation; Byzantine Resilience

Acknowledgement

We begin by offering our deepest gratitude and all praise to Almighty Allah, the Most Merciful and Compassionate, whose divine blessings and infinite mercy enabled us to complete this thesis despite the many challenges we faced.

We extend our profound gratitude to our respected supervisor, Dr. Md. Golam Rabiul Alam, Professor at BRAC University, for his invaluable guidance, unwavering support, and insightful feedback throughout this research. His intellectual rigor and constant encouragement inspired us to strive for excellence even in difficult moments.

We are equally indebted to Dr. Md. Sadek Ferdous, Professor at BRAC University, for his exceptional teachings and steadfast guidance during the most demanding phases of our journey. His wisdom provided us with the clarity and confidence to overcome critical hurdles.

Our sincere appreciation goes to Ms. Anika Tasnim and Md. Yeasin Ali, Research Associates, for their crucial technical support, friendly guidance, and collaborative spirit that greatly enriched our research experience.

We owe an immeasurable debt of gratitude to our beloved parents, whose unconditional love, sacrifices, and unshakeable faith have been the foundation of our entire undergraduate journey. Without their selfless support, this accomplishment would not have been possible.

Finally, we express our heartfelt thanks to our dear friends—Simi, Razin, Oishi, Rafsan, Naser, Sarar, Fardin, and many others—who stood by us with technical assistance and emotional solace, making this challenging journey truly meaningful.

Table of Contents

List of Figures

List of Tables

Nomenclature

The next list describes several symbols & abbreviation that will be later used within the body of the document

ADTCN Adaptive Deep Temporal Context Network — the fraud-detection model

API Application Programming Interface

AUC Area Under the ROC Curve

BOA Butterfly Optimisation Algorithm

CA Certificate Authority (Hyperledger Fabric identity component)

CNN Convolutional Neural Network

DB-BOA Dynamic Butterfly–Billiards Optimisation Algorithm

DP Differential Privacy

DP-SGD Differentially Private Stochastic Gradient Descent

FedAvg Federated Averaging — the baseline aggregation rule

FL Federated Learning

FL-ADTCN Federated Adaptive Deep Temporal Context Network — the proposed framework

FPR False Positive Rate

IID Independent and Identically Distributed

MC Monte Carlo

MCC Matthews Correlation Coefficient

ML Machine Learning

MSP Membership Service Provider (Hyperledger Fabric)

MTTA Multiple Time-Scale Temporal Attention

NPV Negative Predictive Value

PCA Principal Component Analysis

PEC-BC Private Ethereum Consortium Blockchain (the base paper's simulated ledger)

RL Reinforcement Learning

ROC Receiver Operating Characteristic

SDK Software Development Kit

TP, TN, FP, FN True positives, true negatives, false positives, false negatives

ULB Université Libre de Bruxelles (source of the credit-card fraud dataset)

ϵ, δ Differential-privacy budget and failure probability

$\epsilon^\star$ Largest privacy budget at which the on-chain reward ordering still inverts (boundary of the rank-faithful regime)

ϕ_i Shapley contribution weight of organisation i

ρ Spearman rank correlation between the DP and noise-free reward orderings

n, f Number of federation organisations; number of Byzantine organisations tolerated

Chapter 1

Introduction

1.1 Background

The rapid proliferation of digital financial services has created an environment in which fraudulent activity can spread across institutional boundaries faster than any single organisation can detect it. The Association of Certified Fraud Examiners reports that organisations lose approximately five percent of their revenue to fraud each year, with global losses exceeding USD 4.7 trillion annually[24]. In the banking sector specifically, card-not-present fraud, account takeover, and synthetic identity fraud are no longer isolated events — they are coordinated campaigns conducted by actors who exploit the silos between competing financial institutions[24].

The fundamental problem is architectural. Each bank trains its fraud detection model on its own transaction data. A fraudster detected by Bank A immediately redirects to Bank B, which has no knowledge of the prior incident. This institutional blindness is not a failure of technology; it is a consequence of rational competitive behaviour reinforced by regulatory constraint. Data protection laws such as the GDPR in the European Union and the Bangladesh Bank Data Governance Policy prohibit the transfer of raw customer transaction records to third parties without explicit consent — a standard that cannot practically be met in a real-time fraud detection pipeline.

Three distinct barriers have historically prevented banks from building collaborative fraud detection systems. The first is **privacy**: raw transaction data contains personally identifiable information that cannot legally or ethically cross institutional boundaries. The second is **trust**: in a competitive market, no institution is willing to grant a competitor administrative control over a shared detection system, as that competitor could manipulate outcomes, selectively apply rewards, or gain proprietary intelligence from the arrangement. The third is **incentive alignment**: even if the technical and trust barriers were resolved, a profit-maximising institution has no rational reason to invest in improving a fraud detection model whose benefits flow primarily to competitors. Free-riding — contributing a deliberately poor model while benefiting from better models contributed by others — is the Nash equilibrium of any naïve federated system.

This thesis proposes a framework that resolves all three barriers simultaneously through the integration of **Federated Learning** (privacy), a **Hyperledger Fabric permissioned consortium blockchain** (trust), and a **contribution-coupled token incentive mechanism** (incentive alignment). The framework is called **FL-**

ADTCN — Federated Adaptive Deep Temporal Context Network. Its **central contribution** is a *characterisation of the privacy–incentive coupling* of a Shapley-driven, chaincode-enforced reward mechanism on a real permissioned blockchain, together with a concrete fix for it: differential privacy and contribution-based incentives are shown to be in direct tension, and applying the privacy noise to the released contribution channel rather than to the shared model weights restores rank-faithful, differentially-private on-chain rewards at roughly two orders of magnitude lower privacy budget ($\epsilon^\star$ from ≈ 3000 down to ≈ 50). Around this result the thesis provides a characterisation suite that motivates and bounds the mechanism: the decoupling of model accuracy from reward fidelity under privacy noise, the economic isolation of a colluding majority, statistical Byzantine fault tolerance, and the cost–fidelity limits of Shapley attribution as the federation grows.
This central contribution is built on, but is distinct from, an engineering substrate: a federated aggregation layer combining differentially private weight sharing (Gaussian mechanism), Krum robust selection, and exact / Monte-Carlo Shapley contribution attribution, bound to an on-chain token incentive enforced by Hyperledger Fabric chaincode; the Dynamic Butterfly–Billiards Optimisation Algorithm (DB-BOA), which automates the ADTCN detector's hyperparameter tuning and selects the consensus leader; and a deliberately minimal reinforcement-learning (linear Q-learning) policy for *sequential* leader rotation. Rather than claiming algorithmic priority, this work *extends* prior Shapley-on-blockchain incentive systems such as FedCoin [17] and SI-ChainFL [43] with a deployed Hyperledger Fabric implementation and a novel noise→fidelity→reward-error analysis with an accompanying channel-placement result. The contributions are therefore ordered deliberately: the privacy–incentive characterisation is the primary, genuinely new result; the four characterisation experiments that bound it are secondary; and the detector, optimiser, RL policy, and Fabric deployment are the enabling substrate, not co-equal novelties.

1.2 Rationale of the Study

Prabanand and Thanabal [40] published the DB-BOA-ADTCN framework in February 2025, demonstrating that DB-BOA can simultaneously optimise ADTCN hyperparameters and select consensus leader nodes on a simulated Private Ethereum Consortium Blockchain (PEC-BC). Their work established the algorithmic foundation on which this thesis builds.
However, the base paper leaves three gaps that this thesis fills. First, it does not implement federated learning: a single institution's model is optimised and deployed rather than a global model trained collaboratively across organisations. Second, it uses a simulated blockchain (MATLAB-based PEC-BC) rather than a real permissioned network, leaving deployability unvalidated. Third, it provides no mechanism for fairly attributing contribution or distributing incentives across multiple institutions, and no treatment of the privacy machinery that a real multi-bank deployment would legally require. This thesis addresses all three by implementing a federated ADTCN on a real Hyperledger Fabric consortium and by adding a differential-privacy / Krum / Shapley aggregation layer whose Shapley contribution weights drive on-chain token rewards — and then by characterising the previously unexamined interaction between that privacy layer and the fairness of those rewards.
This study is therefore motivated by the need to move from a single-institution, sim-

ulated proof of concept to a privacy-preserving, incentive-aligned, multi-institution system implemented on production-grade blockchain infrastructure, and to understand — rather than assume away — the trade-offs that emerge when privacy, robustness, and economic fairness are required at once.

1.3 Problem Statement

Financial fraud imposes substantial costs on institutions and erodes customer trust. Centralised fraud detection systems require banks to share raw transaction data with a single processing entity, creating privacy risks and single points of failure. Federated Learning resolves the data-sharing problem but introduces new challenges: without a trust-enforcement layer, participants may submit inaccurate or adversarial local model updates; there is no objective mechanism for selecting which node should lead the consensus process; and there is no principled, tamper-evident basis for distributing rewards in proportion to contribution.

Existing blockchain-FL integrations often rely on permissionless or simulated blockchain environments that lack the access control and deterministic execution guarantees required in regulated banking contexts. Moreover, the incentive layer in prior Shapley-on-blockchain work assumes a clean contribution signal: it does not examine how the differential-privacy noise needed for a deployable system corrupts the fidelity of contribution-based rewards once those rewards are written immutably to a ledger. No prior work characterises this privacy–incentive coupling, nor identifies how to restore faithful rewards under a formal privacy guarantee.

This study addresses these gaps by implementing a DB-BOA-ADTCN framework on Hyperledger Fabric that:

1. Uses DB-BOA to select consensus leader nodes by minimising a composite resource objective of computation time, communication cost, and memory size, augmented with a reputation discount factor.

2. Uses DB-BOA to tune ADTCN hyperparameters (convolutional filter count and steps per epoch — a two-dimensional search; the epoch count is fixed, not searched) to obtain a working detector without manual search.

3. Uses **differentially private** weight sharing (Gaussian mechanism) so that the parameters exchanged between institutions carry a formal privacy guarantee.

4. Uses **Krum** robust aggregation to select the most consensus-aligned model and reject outlying, potentially poisoned updates.

5. Uses **Shapley values** (exact at small federation sizes, Monte-Carlo at scale) to attribute each institution's marginal contribution to the global model, replacing naïve equal weighting, and writes these weights to the Fabric ledger.

6. Enforces a token-based incentive structure via `DBBOAContract` chaincode that distributes federation rewards proportionally to the Shapley contribution weights, creating a direct, verifiable link between measured model contribution and economic reward.

7. Characterises and resolves the resulting privacy–incentive coupling, and demonstrates adversarial resilience by showing that the mechanism assigns near-zero reward to a participant that degrades the global model.

1.4 Objective

The specific objectives of this study are:

1. To deploy a Hyperledger Fabric permissioned consortium blockchain with the `DBBOAContract` chaincode that records consensus decisions and enforces token-based incentives for a financial fraud-detection consortium (BankA, BankB, BankC).

2. To implement DB-BOA for ADTCN hyperparameter optimisation — automating selection of the convolutional filter count and steps per epoch (a two-dimensional search; the epoch count is fixed) without manual search (a working detector obtained automatically, though it does not exceed a carefully hand-tuned default).

3. To implement DB-BOA for consensus leader-node selection, minimising the composite resource objective Obf_1 = CT+CC+MS with a reputation discount, and to provide a minimal reinforcement-learning policy for adaptive sequential leader rotation.

4. To implement a privacy-preserving, fairness-aware federated learning pipeline across the three-bank consortium, combining differentially private weight sharing, Krum robust selection, and exact / Monte-Carlo Shapley contribution attribution.

5. To couple the Shapley contribution weights to an on-chain token incentive mechanism so that reward tracks contribution automatically and verifiably.

6. To characterise the privacy–incentive coupling of this mechanism — quantifying the privacy budget at which on-chain rewards remain rank-faithful — and to evaluate its boundaries through the economic isolation of a colluding majority, statistical Byzantine fault tolerance, and the scalability of contribution attribution.

1.5 Methodology in Brief

The framework is evaluated on the public Kaggle ULB Credit Card Fraud Detection dataset (284,807 anonymised transactions; features V1–V28, Amount, and Time; 492 fraudulent cases, 0.17%)[9], partitioned across three simulated banks (BankA 50%, BankB 30%, BankC 20%) by stratified sampling that preserves the global fraud rate in each shard. The research follows a structured six-phase pipeline.
Phase 1 — Leader Node Selection: DB-BOA optimises over the consortium to select the consensus leader by minimising a composite objective of computation time, communication cost, and memory size; a minimal linear-Q reinforcement-learning policy is additionally evaluated for adaptive leader rotation as node reliability drifts.

Phase 2 — Hyperparameter Optimisation: DB-BOA searches the hyperparameter space (convolutional filter count and steps per epoch; the epoch count is fixed) to configure the ADTCN detector automatically, removing the need for manual tuning.
Phase 3 — ADTCN Training: The ADTCN — a temporal one-dimensional convolutional (TCN-family) detector — is trained on each organisation's local data using the DB-BOA-selected hyperparameters.
Phase 4 — Federated Aggregation: At each federation interval the framework applies differentially private weight sharing, Krum robust selection, and exact / Monte-Carlo Shapley contribution weighting; the Shapley weights determine both the global model combination and the on-chain token split. Detection metrics are reported at the deployment operating point, in which weight-channel differential privacy is disabled and privacy is instead enforced on the incentive channel — the configuration the central characterisation shows to be both private and faithful.
Phase 5 — Blockchain Integration: A Hyperledger Fabric network records all transactions, leader elections, consensus rounds, federation rounds, and token balances via the `DBBOAContract` chaincode. Incentive rules (fraud consensus: +10 tokens, latency bonus: +15 tokens, leader success: +10 tokens, penalty: −2 tokens, federation pool: +20 tokens) are enforced deterministically on-chain.
Phase 6 — Evaluation: The federated pipeline is evaluated against FedAvg, FedAvg+Krum, and FedAvg+DP baselines using accuracy, precision, sensitivity, specificity, NPV, FPR, FNR, FDR, F1-score, and MCC. A characterisation suite then bounds the central mechanism: a privacy sweep that locates the rank-faithful reward budget, an economic-isolation experiment against a colluding majority, a statistical Byzantine-fault-tolerance experiment running Krum in its $n \geq 2f + 3$ regime against weight-poisoning attacks, and a scalability study of exact versus Monte-Carlo Shapley attribution as the federation grows.
Figure 1.1 summarises this six-phase pipeline as a workflow.

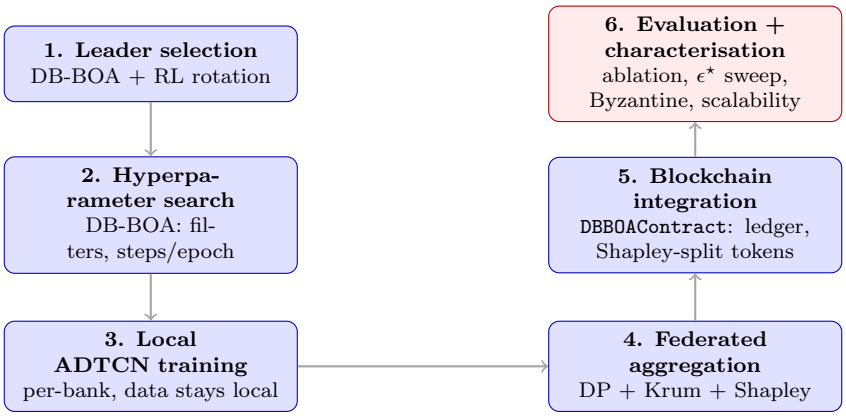


Figure 1.1: Research workflow: the six methodology phases. The highlighted final phase contains the characterisation suite around the central privacy–incentive contribution.

1.6 Summary of Contributions

The contributions of this thesis are ordered deliberately: a primary characterisation result, a secondary suite of characterisation experiments that bounds it, and the engineering substrate that enables both. Rather than claiming algorithmic priority, this work extends prior Shapley-on-blockchain incentive systems such as FedCoin [17] and SI-ChainFL [43] with a deployed Hyperledger Fabric implementation and a noise→fidelity→reward-error analysis.

Primary contribution — the privacy–incentive characterisation and its fix.

- A characterisation of the privacy–incentive coupling of a Shapley-driven, chaincode-enforced reward mechanism on a real permissioned blockchain: differential privacy and contribution-based incentives are shown to be in direct tension, with the noise required for a formal privacy guarantee corrupting the contribution signal that fair on-chain rewards depend on.
- A concrete resolution: applying the privacy noise to the released contribution channel rather than to the shared model weights restores rank-faithful, differentially-private on-chain rewards at roughly two orders of magnitude lower privacy budget ($\epsilon^\star$ from ≈ 3000 down to ≈ 50).

Secondary contributions — the characterisation suite that bounds the mechanism.

- The decoupling of global-model accuracy from reward fidelity under privacy noise: the two degrade at different budgets, so a model that still detects fraud can already be paying dishonest rewards.
- The economic isolation of a colluding majority: the Shapley-weighted reward assigns near-zero contribution weight to adversaries that out-number any statistical defence.
- Statistical Byzantine fault tolerance: Krum, run in the $n \geq 2f + 3$ regime where its theorem holds, rejects every tested weight-poisoning attack while plain averaging collapses.
- The cost–fidelity limits of contribution attribution as the federation grows: exact Shapley is $O(2^n)$, and a Monte-Carlo permutation estimator keeps attribution tractable at larger federation sizes.

Enabling substrate — the engineering on which the results rest.

- An end-to-end deployment on a real Hyperledger Fabric permissioned consortium (BankA, BankB, BankC) whose `DBBOAContract` chaincode records leader elections, consensus rounds, federation rounds, and Shapley-proportional token balances.
- A federated aggregation layer combining differentially private weight sharing (Gaussian mechanism), Krum robust selection, and exact / Monte-Carlo Shapley contribution attribution, bound to the on-chain token incentive.
- The DB-BOA hybrid metaheuristic, which automates the ADTCN detector's hyperparameter tuning and selects the consensus leader, plus a deliberately minimal linear Q-learning policy for sequential leader rotation.

1.7 Scopes and Challenges

This study focuses on financial fraud detection in a consortium banking environment, extending the DB-BOA-ADTCN framework of Prabanand and Thanabal [40] to a Hyperledger Fabric deployment with privacy-preserving, incentive-aligned Federated Learning. The scope covers leader-node selection, model hyperparameter optimisation, differentially private and Byzantine-robust federated aggregation, Shapley-based contribution attribution, on-chain incentive enforcement, and the characterisation of the privacy–incentive coupling — all within a multi-organisation consortium evaluated primarily at three banks and stress-tested at larger sizes for the attribution layer.
Key challenges addressed include:

- **The privacy–incentive tension:** Differential privacy noise, added to protect shared parameters, corrupts the contribution signal that fair rewards depend on; the framework must locate the operating regime and channel placement in which rewards remain faithful under a formal privacy guarantee.
- **Computational overhead:** DB-BOA's population-based search is applied across distinct optimisation tasks per training run. Population sizes and iteration counts were tuned (population: 15–20, iterations: 20–30) to maintain practical runtimes while preserving solution quality.
- **Data heterogeneity:** The banks hold different proportions of training data (50/30/20 split), creating non-IID conditions that stratified partitioning and the contribution-weighted aggregation must accommodate.
- **Adversarial participation:** Both weight-level poisoning (countered by Krum in its $n \geq 2f + 3$ regime) and behavioural collusion (countered by economic isolation through the Shapley-weighted reward) are simulated to test the complementary statistical and economic defences.
- **Blockchain determinism:** Hyperledger Fabric requires all chaincode logic to be fully deterministic. All DB-BOA optimisation and randomness was confined to the off-chain Python layer; only finalised results (leader ID, token deltas, contribution weights, model accuracy) were written to the ledger.
- **Scalability of contribution attribution:** Exact Shapley is $O(2^n)$ and becomes infeasible beyond roughly a dozen organisations; a Monte-Carlo permutation estimator keeps contribution attribution tractable to larger federations. The scalability claim is therefore for *contribution attribution*, not for blockchain throughput or distributed training, which remain outside scope.

Differential privacy (Gaussian mechanism) is already integrated into the weight-sharing and incentive layers; future directions include relaxing the privacy budget with DP-SGD, extending the consortium beyond three organisations in a live deployment, deploying across real multi-host banking nodes, and investigating quantum-resistant cryptographic primitives for long-term ledger integrity.

Chapter 2

Literature Review

2.1 Preliminaries

This thesis examines how a permissioned blockchain can be integrated with Federated Learning (FL) and deep learning to strengthen security, privacy, and fraud detection in financial systems. Centralised approaches to financial fraud detection suffer from single points of failure, data-transparency gaps, and long settlement times. Blockchain reinforces data integrity and enables trusted collaboration across competing financial institutions, while Federated Learning allows those institutions to train a shared detection model without exposing raw transaction data. The resulting framework combines these technologies with the Dynamic Butterfly–Billiards Optimisation Algorithm (DB-BOA) and an Adaptive Deep Temporal Context Network (ADTCN) detector, and binds each participant's measured contribution to an on-chain token incentive. This chapter reviews the four bodies of work the framework rests on — federated learning under data heterogeneity, blockchain for financial security and blockchain-integrated FL, robust/private/fair aggregation, and metaheuristic and reinforcement-learning optimisation for consensus — and isolates the specific gap this research fills.

2.2 Federated Learning

2.2.1 Federated Learning

Federated Learning (FL), introduced by McMahan et al. [5], is a distributed machine-learning paradigm in which model training occurs across multiple participants without centralising raw data. Each participant trains a local model on its private data and shares only model parameters — gradients or weight vectors — with an aggregator, which produces a global model via weighted averaging. The canonical aggregation algorithm, Federated Averaging (FedAvg), weights each participant's contribution proportionally to its dataset size [5].

FL has been applied to sensitive financial and infrastructure settings where data cannot leave the institution that owns it. Li et al. [26] applied FL with a secure deep-learning architecture to detect False Data Injection Attacks in smart grids, validating FL's applicability to safety-critical, privacy-constrained data. Despite its promise, FL is not without challenges: trust in model updates, susceptibility to poisoning attacks from dishonest participants, and the absence of verifiable contri-

bution records remain open problems [36]. This thesis addresses all three through blockchain integration, robust aggregation, and a contribution-based incentive mechanism.

2.2.2 Non-IID Data Heterogeneity

A critical challenge in practical FL deployments is data heterogeneity. Real-world institutional data is typically non-Independent and non-Identically Distributed (non-IID): different banks hold different proportions of fraud cases, reflecting their customer demographics and market exposure. McMahan et al. [5] observed that FedAvg performance degrades under non-IID conditions, and Hsieh et al. [15] characterised this "non-IID quagmire" systematically, showing that the accuracy loss is driven primarily by skewed label distributions across nodes and that the choice of aggregation and normalisation strategy matters more than the optimiser. Li et al. [16] introduced FedProx, a proximal-regularisation technique that constrains local updates to remain close to the global model, stabilising convergence under heterogeneity. This thesis addresses non-IID heterogeneity through stratified partitioning, which preserves the global fraud rate in every institution's shard; proximal methods such as FedProx remain a natural extension for more severely heterogeneous deployments, as discussed in Chapter 4.

2.2.3 Temporal Deep Learning for Sequential Fraud Detection

Financial transactions are inherently sequential, and the detector used in this thesis exploits that structure. Bai et al. [6] showed that Temporal Convolutional Networks (TCNs) — one-dimensional causal, dilated convolutions — match or exceed recurrent architectures on a broad range of sequence-modelling tasks while training faster and avoiding the vanishing-gradient pathologies of RNNs. The Adaptive Deep Temporal Context Network (ADTCN) adopted here is a TCN-family detector whose capacity is tuned per deployment (Chapter 4). Complementary work on imbalanced fraud detection, such as the GNN-based Pick-and-Choose approach of Liu et al. [23], highlights the extreme class imbalance (fraud rates well below 1%) that any financial detector must withstand — a property of the ULB dataset used in this thesis and a motivation for reporting balanced accuracy rather than raw accuracy.

2.3 Blockchain for Financial Security

2.3.1 Consortium Blockchain Architecture

Al-Shaibani et al. [18] proposed a consortium blockchain architecture for a decentralised stock exchange, demonstrating that permissioned networks reduce settlement inefficiency while preserving trading-logic integrity. Their work established that consortium blockchains are well-suited to financial environments where participants are known and access must be controlled — a principle directly adopted here.

Tsoulias et al. [19] extended this direction with a graph-model-based blockchain capable of monitoring and detecting adversarial activity such as dynamic validator

attacks. Their contribution motivated the reputation-aware leader-selection mechanism in this framework.

2.3.2 Hyperledger Fabric for Enterprise Finance

Li et al. [31] designed Fabric-SCF, a Hyperledger Fabric-based secure storage and access-control scheme using distributed consensus and attribute-based access control, demonstrating high throughput and dynamic fine-grained access in supply-chain-finance scenarios. Their use of Fabric's permissioned architecture and chaincode-enforced rules directly informs this thesis's deployment environment. Wang and Wang [28] demonstrated that blockchain-IoT data sharing combined with edge computing can monitor fraudulent financial transactions, identifying intensive computation handling as a remaining challenge — one this thesis addresses through off-chain model training with on-chain result recording.
The choice of Hyperledger Fabric over other blockchain platforms is architecturally motivated. Fabric is permissioned: only enrolled organisations with valid X.509 certificates issued by a Fabric Certificate Authority can participate, a hard requirement in regulated financial environments. Unlike Ethereum's public network, which allows anonymous participation, Fabric's consortium model aligns with banking compliance requirements. Furthermore, Fabric does not use energy-intensive proof-of-work; its Raft-based ordering service provides deterministic, low-latency consensus, critical for financial settlement, and its CouchDB world state supports the JSON queries required for audit and reporting.

2.3.3 Smart Contract Enforcement

The role of smart contracts in enforcing financial rules without trusted intermediaries is a recurring theme. Ying et al. [41] demonstrated in BIT-FL that blockchain-enabled smart contracts can enforce verifiable FL incentive rules, though their implementation used a simulated environment rather than a real Fabric network. Truong et al. [35] further validated that smart-contract-backed trust mechanisms are essential for AI-generated-content trading in decentralised environments, supporting the use of Fabric chaincode for incentive enforcement here.

2.4 Blockchain-Integrated Federated Learning

2.4.1 Security and Trust in Blockchain-FL Systems

Yang et al. [36] proposed on-chain model aggregation and an incentive mechanism for industrial-IoT federated learning, demonstrating that a blockchain can verify global-model updates and detect malicious alterations. Their work validates the architectural principle adopted here: computation off-chain, verification on-chain. Hussin Chowdhury et al. [32] showed that integrating federated machine learning into a blockchain crowdfunding ecosystem increases both security and transparency, supporting the adoption of FL for financial consortium settings. Saveetha et al. [34] proposed an integrated FL-and-blockchain framework with optimal miner selection for DDoS-attack detection, using metaheuristic optimisation for miner selection — a conceptual predecessor to this thesis's optimiser-driven leader selection. Their

system, however, does not address contribution-fair aggregation or couple economic incentives to a measured contribution signal.

2.4.2 Incentive Mechanisms

Blockchain-based incentive mechanisms reward reliable and resource-efficient contributors to sustain participation. Zhao et al. [37] introduced the Long-Term Proof-of-Contribution algorithm for blockchain-enabled federated learning, demonstrating that incentive structures can sustain honest participation across many rounds. A more principled line of work grounds rewards in cooperative game theory: FedCoin (Liu et al. [17]) is a peer-to-peer payment system that allocates rewards by Shapley value, and SI-ChainFL (Zhao et al. [43]) couples Shapley-incentivised rewards with a secure FL pipeline for high-speed-rail data sharing. These works establish that Shapley-value reward allocation on a blockchain is an established idea, not a novelty of this thesis.

What the prior incentive literature does *not* examine is the interaction between the privacy noise required for a deployable FL system and the fidelity of the contribution signal that those rewards are computed from. Because rewards in this thesis are written immutably to a Fabric ledger, even small contribution-ranking errors become permanent, mispaid incentives. Characterising that interaction — rather than proposing a new reward rule — is the contribution this thesis builds on top of the FedCoin/SI-ChainFL line, and is developed in Section 2.5.4.

2.5 Robust, Private, and Fair Federated Aggregation

Three techniques underpin the aggregation layer of this thesis, and each rests on an established body of work.

2.5.1 Differential Privacy

Differential privacy (DP), formalised by Dwork et al. [2], calibrates noise to the *sensitivity* of a computation: the Gaussian mechanism adds noise $\mathcal{N}(0, \sigma^2)$ with $\sigma = C\sqrt{2\ln(1.25/\delta)}/\epsilon$ after clipping to an L_2 bound C, yielding an (ϵ, δ) guarantee on a released statistic. Two refinements are directly relevant. Chaudhuri et al. [3] introduced *output perturbation* for differentially private empirical risk minimisation — adding calibrated noise to a low-dimensional released output rather than to a high-dimensional parameter vector — which is the principle this thesis exploits to make the incentive channel private at a tractable budget. Andrew et al. [21] showed that the clipping bound C can be learned adaptively rather than fixed, reducing the noise needed for a target ϵ; McMahan et al. [7] demonstrated DP-SGD at scale for recurrent language models. A recurring lesson from this literature, central to Chapter 5, is that the noise required to protect a d-dimensional release grows with $\sqrt{d}$, so the dimensionality of the protected channel — not DP itself — sets the practical privacy budget.

2.5.2 Byzantine-Robust Aggregation

FedAvg is fragile to adversarial updates: a single poisoned vector can dominate the average if its norm is large enough. Blanchard et al. [4] introduced Krum, a Byzantine-robust aggregation rule that scores each update by the summed squared L_2 distance to its $n - f - 2$ nearest neighbours and selects the most consensus-aligned update, provably tolerating up to f Byzantine participants when $n \geq 2f + 3$. This thesis uses Krum in exactly that regime to defend the aggregation layer against weight-level poisoning, while noting its boundary: a colluding *majority* ($> f$) defeats any statistical filter, which is where the economic incentive layer provides a complementary defence.

2.5.3 Contribution Attribution via Shapley Values

Fair reward allocation requires quantifying how much each participant's data improves the global model. The Shapley value, from cooperative game theory, assigns each participant its average marginal contribution across all coalitions and is the unique attribution satisfying efficiency, symmetry, and the null-player axiom. Wang et al. [12] adapted it to federated learning (FedSV), and Ghorbani and Zou [11] developed Data Shapley together with Monte-Carlo and truncated estimators that make attribution tractable when the exact $O(2^n)$ computation is infeasible. This thesis uses an exact Shapley value at small federation sizes and a Monte-Carlo permutation estimator to keep contribution attribution feasible as the federation grows.

2.5.4 The Privacy–Incentive Tension and Closest Related Work

Combining DP, Krum, and Shapley exposes a tension that the prior literature treats only in one direction. Li et al. [39] use Shapley values to *guide* where DP noise is injected (Shapley $\rightarrow$ noise), improving the privacy–utility trade-off of the trained model. The coupling this thesis characterises is the reverse: how the DP noise added for privacy degrades the *fidelity* of the Shapley signal and therefore corrupts the on-chain reward (noise $\rightarrow$ Shapley $\rightarrow$ reward error). Closely related, Jaramillo-Velez et al. [42] study private and robust contribution evaluation in FL, and Commey et al. [38] design a Bayesian, poison-resilient incentive mechanism; Fraboni et al. [22] formalise free-rider attacks that any contribution-based reward must resist. Against this backdrop, the contribution of this thesis is a *characterisation*: it shows that contribution-based incentives and weight-channel differential privacy are in direct tension, and that applying the privacy noise on the released, low-dimensional contribution channel (output perturbation, after Chaudhuri et al. [3]) rather than on the high-dimensional weight channel restores rank-faithful, DP-protected on-chain rewards at roughly two orders of magnitude lower privacy budget. This extends the Shapley-on-blockchain incentive line (FedCoin [17], SI-ChainFL [43]) by bounding *when* such a mechanism remains both private and economically faithful on a deployed Fabric network — it does not claim a new reward algorithm.

2.6 Leader Selection and Consensus Optimisation

Leader block selection and consensus efficiency are central concerns in permissioned blockchain systems for high-frequency financial transactions. Zhuang et al. [14] proposed Proof of Reputation, a consensus protocol in which high-reputation nodes are selected as block leaders, increasing throughput while maintaining security. This informs the reputation-aware leader selection in this framework, where a reputation discount factor is applied to the optimiser's objective so that previously reliable nodes become more competitive. Nourmohammadi and Zhang [27] proposed an on-chain governance model based on Particle Swarm Optimisation for reducing blockchain forks, establishing a precedent for metaheuristic-based consensus management. Machhale et al. [33] demonstrated that combining blockchain with machine learning enhances data security and privacy, but, like most works in this area, treated the consensus layer and the application layer as independent concerns.

2.6.1 Reinforcement Learning for Adaptive Leader Selection

Where a metaheuristic optimiser selects a leader from a static snapshot of node fitness, a *sequential* policy can adapt as node reliability drifts at runtime. Q-learning, introduced by Watkins and Dayan [1] and developed in the standard treatment of Sutton and Barto [8], learns an action-value function online from reward feedback without a model of the environment. This thesis includes a deliberately minimal linear-Q leader-rotation policy that, in simulation, matches the optimiser on reward while adapting to nodes whose reliability degrades over time. It is positioned as a secondary, complementary component to the privacy–incentive contribution, not as a novel RL method.

2.7 Metaheuristic Optimisation in Blockchain-AI Systems

2.7.1 Butterfly Optimisation Algorithm

The Butterfly Optimisation Algorithm (BOA), introduced by Arora and Singh [10], models the foraging behaviour of butterflies: each candidate solution emits a "fragrance" proportional to its fitness, attracting other solutions toward high-fitness regions. BOA shows strong exploration but is prone to premature convergence when population diversity is lost.

2.7.2 Dynamic Butterfly and Billiards Optimisation

Two extensions address BOA's weaknesses and supply the components of the hybrid used here. Tubishat et al. [20] proposed the Dynamic Butterfly Optimisation Algorithm (DBOA), which augments BOA with a Local Search Adaptive Mutation mechanism that restores diversity when the population converges prematurely, improving exploitation at the cost of slower convergence on high-dimensional problems. Independently, Givi and Hubálovská [29] introduced the Billiards Optimisation Algorithm, a game-based metaheuristic whose collision-and-rebound dynamics provide

an alternative, momentum-driven exploration operator. Together these motivate a hybrid that switches between butterfly-style and billiards-style moves according to population state.

2.7.3 DB-BOA: The Hybrid Algorithm

Prabanand and Thanabal [40] introduced DB-BOA, a Dynamic Butterfly–Billiards hybrid governed by an adaptive switching criterion: at each iteration t a uniform random variable rand $\sim \mathcal{U}[0,1]$ is compared to the ratio $\text{bestfit}_t/\text{worstfit}_t$ of the best and worst objective values in the current population. When the ratio approaches 1 (population converging) the exploitation operator dominates; when it is small (population diverse) the exploration operator dominates. This balances exploration and exploitation without manual schedule tuning. In their base paper, Prabanand and Thanabal applied DB-BOA on a *simulated* private-Ethereum consortium to two tasks — hyperparameter tuning of ADTCN and leader-node selection — reporting reduced consensus latency relative to random selection. This thesis re-implements DB-BOA for the same two roles (hyperparameter tuning and leader selection) on *real* Hyperledger Fabric infrastructure; the federated aggregation weights, by contrast, are produced by Shapley attribution (Section 2.5), not by DB-BOA.

2.8 Summary of Related Work and Research Gap

Table 2.1 summarises the prior-work landscape and the specific gap this research fills.

The specific gap this research fills is: *prior systems either allocate Shapley-based blockchain incentives while assuming a clean contribution signal, or study differential privacy and robust aggregation in isolation; none characterise how the privacy noise required for a deployable FL system corrupts the fidelity of contribution-based, immutable on-chain rewards, nor identify the released-channel perturbation that restores them — on real Hyperledger Fabric infrastructure.*

2.9 Summary of Key Findings

The reviewed literature highlights three pillars that underpin blockchain-integrated financial-security systems. First, **permissioned consortium blockchains** — exemplified by Hyperledger Fabric — provide the access control, immutability, and smart-contract capability needed to enforce financial rules without intermediaries [18], [31], [40]. Second, **Federated Learning** lets institutions collaboratively train fraud detectors without sharing raw data, though trust, robustness, and verifiable contribution require an external verification layer [5], [26], [36] and dedicated aggregation machinery — differential privacy [2], [3], Byzantine-robust selection [4], and Shapley contribution attribution [11], [12]. Third, **optimisation and incentive alignment** are essential for deployment: metaheuristics such as DB-BOA automate leader selection and hyperparameter tuning [27], [40], reinforcement learning adapts leader rotation online [1], and Shapley-based on-chain incentives sustain honest participation [17], [37], [43]. The open problem that motivates this thesis sits at the intersection of the second and third pillars: the privacy noise that

Table 2.1: Summary of Related Work and Identified Research Gap

Prior Work Category	**What It Achieves**	**What It Misses**
Centralised fraud detection with ML	High accuracy, fast training	Privacy loss, single point of failure, no multi-party collaboration
Federated learning for fraud detection	Privacy-preserving collaboration	No verifiable contribution record; FedAvg fragile to poisoning and ignores model quality; no blockchain audit trail
Blockchain for financial data sharing	Transparency and immutability	Consensus layer and application layer treated independently; no ML; no FL
Shapley-on-blockchain incentives (FedCoin, SI-ChainFL)	Game-theoretically fair, on-chain rewards	Privacy noise vs. contribution-fidelity interaction left uncharacterised; rewards assumed clean
Robust / private FL (Krum; DP; Shapley-guided noise)	Byzantine tolerance or DP utility, studied in isolation	The *reverse* privacy→incentive coupling, and which channel to perturb, not examined
Metaheuristic optimisation for blockchain	Improved consensus efficiency	Applied to one objective only; not combined with FL fraud detection
DB-BOA base paper (Prabanand & Thanabal, 2025) [40]	DB-BOA for hyperparameter tuning and leader selection on a *simulated* consortium	No federated learning; no contribution-fair incentive; no real Hyperledger Fabric implementation
This thesis (FL-ADTCN)	**Federated ADTCN on real Hyperledger Fabric with DP + Krum + Shapley aggregation and Shapley-weighted, chaincode-enforced on-chain incentives, plus DB-BOA/RL leader selection**	**Characterises (rather than ignores) the privacy–incentive coupling that the above prior work leaves open**

makes FL deployable and the contribution signal that makes incentives fair are in tension, and that tension — not a new algorithm — is what this thesis characterises and resolves at the channel level, implemented end-to-end on a real Hyperledger Fabric consortium and evaluated on the public ULB Credit Card Fraud dataset [9].

Chapter 3

Requirements, Impacts and Constraints

3.1 Final Specifications and Requirements

This project implements the DB-BOA-ADTCN financial security framework, extending the work of Prabanand and Thanabal [40] onto a Hyperledger Fabric permissioned consortium blockchain with a full Federated Learning pipeline across a three-bank consortium (BankA, BankB, BankC).

Technical and Methodological Requirements

The framework adopts Federated Learning (FL) to enable decentralized fraud detection model training where raw transaction data remains local to each participating bank, preventing centralized data exposure.
A Hyperledger Fabric permissioned consortium blockchain provides immutability, auditability, and controlled participation, with access restricted to authenticated consortium members through Fabric's certificate authority (CA) infrastructure.
Fabric's built-in ordering service (RAFT-based) provides deterministic, low-latency consensus suited to high-frequency financial transaction environments, avoiding the computational waste of public blockchain consensus mechanisms.
Fabric chaincode (smart contracts written in Node.js) manages leader election records, fraud verdict logging, consensus round tracking, federation round aggregation, and token-based incentive allocation across all consortium nodes.
The Dynamic Butterfly–Billiards Optimization Algorithm (DB-BOA) is applied to two tasks: (1) ADTCN hyperparameter optimisation — tuning the convolutional filter count and steps per epoch (a two-dimensional search; the epoch count is fixed); and (2) consensus leader-node selection — minimising computation time, communication cost, and memory size with a reputation discount. The federated aggregation layer is handled separately by three dedicated mechanisms: differentially private weight sharing (Gaussian mechanism), Krum robust model selection, and exact / Monte-Carlo Shapley-value contribution attribution, with the Shapley weights driving the on-chain token incentive.
Adaptive Deep Temporal Context Networks (ADTCN) — realised as a temporal one-dimensional convolutional network over a sliding window of recent transactions, with global max-pooling over time and a linear fraud/normal classifier — serve

as the core detection model, capturing temporal patterns in financial transaction sequences.
A token-based incentive structure enforced on-chain rewards accurate fraud verdicts (+10 tokens), low-latency consensus (+15 tokens), and successful leader rounds (+10 tokens), while penalizing incorrect verdicts and failed rounds (-2 tokens each) and distributing a federation participation pool (+20 tokens shared in proportion to the Shapley contribution weights).

Software and Resource Requirements

- **Machine Learning Implementation:** Python 3 with **PyTorch** for the **ADTCN** detector and **NumPy/scikit-learn** for the **DB-BOA** optimiser, the DP/Krum/Shapley federation layer, and the evaluation metrics.
- **Blockchain Infrastructure: Hyperledger Fabric 2.5** (Docker test-network) for the permissioned consortium blockchain, with the `fabric-network` **2.2 Node.js SDK** (wallet-based) for application-layer interaction.
- **Smart Contract Logic: Node.js chaincode** (`DBBOAContract`) using `fabric-contract-api/fabric-shim` 2.5, deployed as a Chaincode-as-a-Service on the consortium peers, implementing all on-chain state management, incentive enforcement, and ledger operations deterministically.
- **Incentive and Security Mechanism:** Token-based scoring system enforced via chaincode, with reputation deltas applied on-chain per consensus round outcome.
- **Dataset:** the public **ULB Credit Card Fraud** dataset (284,807 transactions; features V1–V28, Amount, and Time; 492 fraudulent cases, 0.17%), partitioned 50/30/20 across the three consortium banks by stratified sampling that preserves the global fraud rate in each shard.
- **Development Environment:** Local workstation utilizing **Python 3**, **Node.js 18**, and **Docker** for Fabric network deployment; no specialized mining hardware required.

3.2 Societal Impact

The framework improves financial security and trust for individuals and SMEs through Hyperledger Fabric's consortium consensus, federated learning for privacy-preserving fraud detection, and on-chain incentives for honest validator participation. By keeping raw transaction data local to each bank and enforcing verdict rules through tamper-evident chaincode, it enables robust fraud detection without requiring institutions to surrender data sovereignty or rely on centralized intermediaries. Widespread adoption across banking consortia could lower systemic fraud risk — reducing the blast radius of any single compromised node — and promote economic inclusion in emerging markets such as Bangladesh by making enterprise-grade fraud detection accessible without prohibitive infrastructure costs. The framework's permissioned architecture also aligns naturally with financial data protection regulations, as consortium membership and data access are cryptographically controlled.

3.3 Environmental Impact

The framework maintains a low environmental footprint by running entirely on existing commodity hardware within a permissioned Hyperledger Fabric network, avoiding the energy-intensive proof-of-work mining of public blockchains. Fabric's RAFT-based ordering service and deterministic chaincode execution require minimal computational resources per transaction. DB-BOA's efficient metaheuristic search reduces unnecessary model evaluations during optimization, and federated learning performs gradient updates locally at each bank, eliminating large-scale data transfers to cloud servers. The token penalty mechanism discourages wasteful re-proposals and failed consensus rounds, further reducing redundant computation. Overall, the framework maximizes the utility of existing banking infrastructure, extends device lifecycles through software-based security enhancement, and promotes sustainable computing practices in the financial sector.

3.4 Ethical Issues

The framework prioritizes ethical data handling throughout its design. Hyperledger Fabric's permissioned architecture restricts network participation to authenticated consortium members (BankA, BankB, BankC), ensuring that transaction records and model outputs remain confidential and inaccessible to unauthorized parties. Federated learning preserves each bank's data sovereignty by keeping raw transaction records local — only differentially private, contribution-weighted model parameters are shared across organizations. The chaincode enforces incentive and penalty rules transparently and without discretion, providing an auditable on-chain record of all consensus decisions and token allocations that any consortium member can verify. The reputation system promotes fair and honest participation without exploiting resource-constrained members, as penalties are proportional and bounded. All processing and ledger state remain within the consortium's controlled infrastructure, aligning with privacy-by-design principles and giving participants full oversight of how their contributions are recorded and rewarded — balancing strong fraud detection capability with respect for privacy, transparency, and institutional accountability.

3.5 Standards

The proposed framework adheres to established best practices and emerging standards in permissioned blockchain and privacy-preserving machine learning. The Hyperledger Fabric network follows the Hyperledger Foundation's recommended patterns for enterprise consortium deployments, including certificate authority-based identity management, channel-level data isolation, and deterministic chaincode execution requirements. Chaincode implementation follows Fabric's determinism rules — no `Math.random()` or `new Date()` in state-mutating functions; timestamps are read from `ctx.stub.getTxTimestamp()` to ensure identical execution across all endorsing peers. The federated learning pipeline complies with privacy principles in IEEE 2986-2023 (Recommended Practice for Privacy and Security in Federated Machine Learning), ensuring local data processing and minimal parameter sharing. The

DB-BOA optimization and ADTCN training pipeline follows the methodology established in Prabanand and Thanabal [40], providing a reproducible baseline for comparative evaluation. Open-source tools (Hyperledger Fabric, Python, Node.js) promote reproducibility and community-vetted development standards.

3.6 Project Management Plan

The project follows a structured, phase-driven execution plan to ensure timely delivery and systematic validation of the proposed framework.

Schedule

- **Literature Review & Problem Analysis** Review of the base paper [40] and related blockchain-FL financial security literature; identification of gaps addressed by migrating to Hyperledger Fabric and adding a federated learning pipeline.
- **Framework Design & Architecture Development**
  Design of the DB-BOA-ADTCN architecture on Hyperledger Fabric, including the three-bank consortium topology, chaincode data model, DB-BOA task decomposition (hyperparameter tuning and leader selection), and the DP/Krum/Shapley federated aggregation layer.
- **Model Development & Integration**
  Implementation of the ADTCN model in Python, the DB-BOA optimizer, the Fabric chaincode (`DBBOAContract`), and the Node.js API server bridging the Python simulation layer with the on-chain ledger.
- **Optimization & Security Enhancement**
  Integration of the two DB-BOA optimisation phases (hyperparameter tuning and leader selection), the DP/Krum/Shapley federated aggregation layer, the token-based on-chain incentive mechanism, and the adversarial resilience simulations (weight-level poisoning countered by Krum, and a colluding-majority scenario countered by economic isolation).
- **Experimental Evaluation & Analysis**
  Performance evaluation of the federated pipeline against FedAvg, FedAvg+Krum, and FedAvg+DP baselines using accuracy, MCC, FPR, NPV, sensitivity, specificity, F1-score, and throughput/latency metrics, followed by the privacy–incentive characterisation suite.
- **Documentation & Thesis Preparation**
  Result interpretation, comparison with base paper findings, discussion of Fabric-specific implementation decisions, identification of limitations, and formulation of future work directions.

Budget Considerations

- No licensing costs are incurred due to the use of open-source frameworks (Hyperledger Fabric, Python, Node.js, Docker).

- No additional hardware expenditure is required; the Fabric network and all experiments are conducted on standard computing resources via Docker containers.
- Overall cost is kept minimal, making the framework suitable for small and medium financial institutions and emerging economies.

Resource Management

- Development effort is distributed across blockchain network configuration, Python-based ML and optimization implementation, Node.js chaincode development, and experimental evaluation tasks.
- Modular code organization (separate `models/`, `algorithms/`, `blockchain/`, `utils/` packages) and a shared configuration file (`config.py`) ensure consistent parameterization and smooth integration across all subsystems.

3.7 Risk Management

Several technical and operational risks were identified during the development of the DB-BOA-ADTCN framework on Hyperledger Fabric. Corresponding mitigation strategies were incorporated into the system design to ensure robustness, security, and scalability. Table 3.1 summarizes the key risks, their potential impacts, and the adopted mitigation measures.
Overall, the risk mitigation mechanisms are embedded directly into the system architecture — through on-chain chaincode enforcement, off-chain optimization constraints, and deterministic Fabric deployment practices — ensuring that security, privacy, and performance are core design principles rather than post-deployment considerations.

3.8 Economic Analysis

The framework offers clear economic advantages for financial institutions and consortium members by automating fraud detection and consensus enforcement through Fabric chaincode, eliminating the need for expensive third-party verification intermediaries and reducing per-transaction settlement costs. The token-based incentive structure lowers participation barriers — organizations earn rewards proportional to their contribution quality rather than requiring large capital stakes. Federated learning eliminates the data-transfer and cloud-processing costs associated with centralized fraud detection services, as model training occurs locally on each bank's existing infrastructure. Development relies entirely on open-source tools (Hyperledger Fabric, Python, Node.js, Docker) running on commodity hardware, resulting in minimal upfront and ongoing maintenance costs. For SMEs and financial institutions in emerging economies, the combination of fraud prevention, reduced intermediary fees, and infrastructure reuse provides strong cost-benefit returns — making enterprise-grade, privacy-preserving fraud detection accessible without prohibitive investment.

Table 3.1: Key risks and mitigations

Risk	**Impact**	**Mitigation (as implemented)**
Model poisoning by an organisation	Corrupted global model	Krum consensus-aligned selection ($n \geq 2f + 3$) + Shapley down-weighting + on-chain token/reputation penalty
Privacy leakage via shared weights	Re-identification risk	Differentially private (Gaussian-mechanism) weight sharing
Privacy noise corrupting incentives	Mispaid, immutable on-chain rewards	Output-channel perturbation of the contribution statistic rather than the weights, restoring rank-faithful rewards at lower budget
Non-deterministic chaincode	Endorsement mismatch / consensus failure	All optimisation and randomness kept off-chain; only finalised results written on-chain
Extreme class imbalance (0.17%)	Trivial all-normal classifier	Weighted cross-entropy loss; MCC and ROC-AUC as primary metrics
Single-source dataset	Limited ecological validity	Stated as a limitation; future work uses real multi-bank streams

Chapter 4

Proposed Methodology

4.1 Design Process or Methodology Overview

Figure 4.1 presents the end-to-end methodology of the proposed framework: the three-bank consortium and its local ADTCN detectors, the two optimisation roles of DB-BOA, the privacy-preserving federation pipeline, and the Hyperledger Fabric layer that records every decision and enforces the token incentive. The highlighted path through the Shapley attribution, the output-perturbation privacy channel, and the chaincode-enforced reward marks the privacy–incentive coupling that constitutes the primary contribution of this thesis (Section 4.1.5). The subsections that follow describe each component in turn.

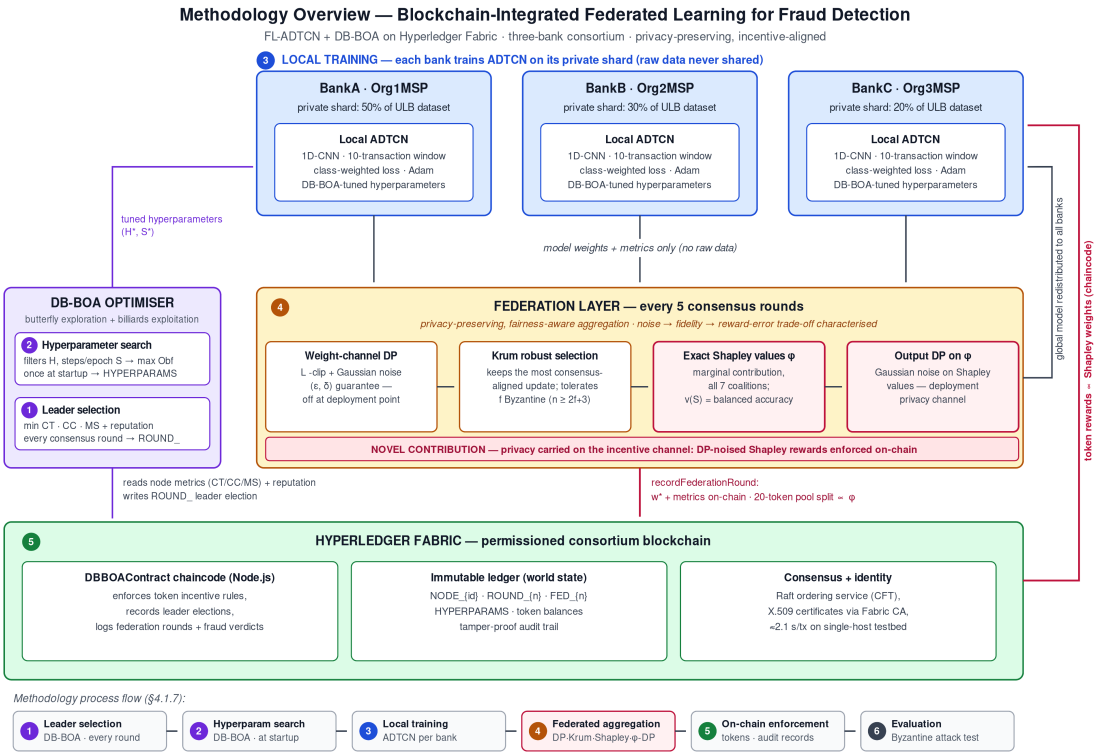


Figure 4.1: End-to-end methodology overview. Three banking organisations (Org1–Org3 MSPs) train local ADTCN detectors on private 50/30/20 shards of the ULB dataset and share only model weights and metrics. DB-BOA performs hyperparameter search (once, at startup) and reputation-discounted leader selection (every consensus round). Each federation round applies differentially private weight sharing (disabled at the deployment operating point), Krum robust selection, and exact Shapley attribution, with output-perturbation DP on the Shapley values serving as the deployment privacy channel. The `DBBOAContract` chaincode on Hyperledger Fabric records all decisions immutably and splits the 20-token federation pool in proportion to the DP-noised Shapley weights; the highlighted (crimson) path marks this privacy–incentive coupling, the primary contribution. Numbered badges correspond to the six methodology phases of Section 4.1.7.

4.1.1 Federated Learning: Decentralized Model Training with Privacy Preservation

Federated Learning (FL) represents a paradigm shift in distributed intelligence—one that is particularly suited to the financial sector, where regulatory compliance and confidentiality are paramount [32]. In traditional financial analytics, models are trained using centrally stored customer data, which raises severe privacy and legal concerns under frameworks such as GDPR, PCI-DSS, and the Bangladesh Bank Data Governance Policy [25]. FL addresses this by allowing institutions—such as banks, credit unions, and regulatory agencies—to collaboratively train a shared fraud-detection model without sharing their customers' raw transaction data [32]. Each financial entity operates as an independent node, locally training an instance of the Adaptive Deep Temporal Context Network (ADTCN) on its internal transaction dataset. After training, the node transmits only model parameters (weights

and gradients) to the blockchain for global aggregation [27]. This process ensures that sensitive details—account numbers, transaction histories, or customer metadata—never leave institutional boundaries, thus maintaining compliance and protecting competitive privacy [30].
In this implementation, the consortium consists of three banking organisations—BankA (Org1MSP), BankB (Org2MSP), and BankC (Org3MSP)—partitioned from the Kaggle ULB Credit Card Fraud Detection dataset in a 50/30/20 split respectively. Each bank trains its local ADTCN on its private shard; only model weight vectors and performance metrics cross institutional boundaries via the Hyperledger Fabric ledger.

Applications and usefulness:

- *Inter-bank fraud analytics:* Federated ADTCN models can detect coordinated fraud patterns across banks without exposing proprietary data [13].
- *Credit scoring collaboration:* Multiple financial institutions can build a common model for creditworthiness assessment while preserving customer confidentiality [13].
- *Cross-border compliance:* Enables secure model sharing between different jurisdictions under privacy laws [25].

Federated Learning workflow within the consortium system:

1. Each validator node (BankA, BankB, BankC) trains a local ADTCN model using its private transaction data shard [32].
2. The model parameters—not raw data—are transmitted to the Hyperledger Fabric ledger via the DBBOAContract chaincode [27]; the parameters are differentially privatised (Gaussian mechanism [2]) before sharing.
3. The federation manager applies **Krum** robust selection [4] to choose the most consensus-aligned update, then computes **exact Shapley values** [12] to attribute each institution's marginal contribution to the global model — replacing the naive equal-weighting of standard FedAvg with a measured contribution weight.
4. The global model is computed and redistributed to all nodes, improving detection accuracy collaboratively [32].
5. Federation rewards (tokens) are distributed proportionally to the Shapley contribution weights, creating a direct link between measured model contribution and economic reward.

The financial sector is characterised by multi-institutional collaboration, requiring transparency, auditability, and immutability—core features provided by blockchain. While the base paper employed a Private Ethereum Consortium Blockchain (PECBC), this research implements an equivalent architecture through Hyperledger Fabric (HLF), which offers modular, enterprise-ready permissioned blockchain infrastructure with deterministic consensus and certificate-based access control.

4.1.2 Hyperledger Fabric: The Infrastructure Layer

Hyperledger Fabric provides the fundamental decentralised architecture required for secure multi-party collaboration in this framework. As a production-grade permissioned consortium blockchain maintained by the Linux Foundation, Fabric restricts network participation to pre-authorised organisations — in this implementation, three banking institutions (BankA, BankB, BankC) — authenticated through Fabric's built-in Certificate Authority (CA) infrastructure. Unlike public blockchains, Hyperledger Fabric ensures that ledger data remains entirely within the consortium, providing the privacy of a permissioned network while maintaining the decentralised trust guarantees of a distributed ledger.

Functional Role within the Framework:

- **Immutable Transaction Ledger:** Provides a tamper-proof, chronological record of all network activities — including fraud verdicts, leader elections, consensus rounds, federation rounds, per-organisation model performance records, and token balances — ensuring that no committed state can be altered or deleted.
- **Smart Contract Environment:** Executes the `DBBOAContract` chaincode, written in Node.js, which automates the enforcement of all consortium rules including incentive allocation, leader election recording, and federated aggregation logging — without human intervention and with full deterministic reproducibility across all endorsing peers.
- **Certificate-Based Identity Management:** Implements a robust identity system where each participant organisation is authenticated via X.509 certificates issued by the Fabric CA, allowing granular control over transaction submission, endorsement rights, and ledger read/write permissions per channel.
- **High-Integrity Consensus:** Operates a Raft-based ordering service to synchronise ledger state across all consortium nodes deterministically and with low latency, eliminating the risk of a single point of failure without the computational waste of proof-of-work mechanisms.

Applications and Relevance:

- *Transparent Auditing:* All administrative actions — fraud verdicts, consensus round outcomes, token allocations, and federation weight assignments — are logged on-chain via the `DBBOAContract`, allowing any consortium member or regulator to perform real-time audits of the system's operational integrity.
- *Data Integrity Verification:* The Fabric ledger acts as the authoritative source of truth for verifying the origin, timing, and outcome of every consensus round and federation event, with each record keyed by a deterministic document ID (e.g., `NODE_{id}`, `ROUND_{num}`, `FED_{num}`).
- *Resilient Network Architecture:* By distributing the ledger across the consortium's peer nodes deployed via Docker, the system remains operational even if individual peers or local servers experience hardware failure or network disruption.

By utilising Hyperledger Fabric as the consortium infrastructure, the framework provides a permissioned, deterministically executable environment with certificate-based access control that balances the transparency of blockchain with the strict confidentiality and access-control requirements of regulated financial institutions. (Measured consensus latency on the single-host test deployment is $\approx 2.1\,\mathrm{s}$ per transaction; production-grade throughput would require a tuned multi-host network and is out of scope — see Limitations.)

4.1.3 DB-BOA: Dynamic Butterfly–Billiards Optimization Algorithm

Background and Conceptual Foundation. The Dynamic Butterfly–Billiards Optimization Algorithm (DB-BOA) is the hybrid optimization core that ensures the efficiency, security, and stability of the blockchain network. It merges two biologically and physically inspired algorithms — Dynamic Butterfly Optimization Algorithm (DBOA) and Billiards Optimization Algorithm (BOA) — combining the adaptive exploration of the former with the deterministic precision of the latter.
DBOA mimics the foraging and mating behaviour of butterflies. Each butterfly's "fragrance" represents the fitness of a solution, guiding others toward global optima. This ensures global exploration of the solution space.
BOA draws inspiration from the physical laws of billiards, where the motion and collision of balls are modelled mathematically to refine solutions within a constrained search space.
By integrating both, DB-BOA dynamically balances exploration and exploitation, reducing the risk of getting trapped in local minima while optimising the consensus and learning parameters of a blockchain-driven learning system.
Two Optimisation Roles in This Framework. In the financial consortium, DB-BOA acts as an intelligent optimisation layer operating across two tasks:

- *Hyperparameter Optimisation:* Automatically searches a two-dimensional space — the convolutional filter count H_{nD} and steps per epoch S_{eD} (the epoch count is fixed, not searched) — maximising a bounded fraud-detection fitness on a validation subsample. Runs once at system startup; results stored on-chain under the `HYPERPARAMS` record.

- *Leader Block Selection:* Selects the validator node (leader) responsible for block proposal by minimising computation time, communication cost, and memory size, augmented with a reputation discount. Runs at the beginning of every consensus round.

The federated aggregation weights are *not* computed by DB-BOA. They are obtained from a dedicated federation layer — differentially private weight sharing, Krum robust selection, and exact Shapley contribution attribution — described in Section 4.1.5. (A DB-BOA-based aggregation mode exists in the codebase but is disabled by default in favour of the Shapley path.)
Financial Domain Applications:

- *High-Frequency Trading (HFT) Platforms:* DB-BOA ensures ultra-low latency transaction validation and leader rotation, reducing delays in block finalisation.

- *Inter-bank Transaction Gateways:* Minimises communication cost and verification time between consortium nodes during peak transactional loads.
- *Risk Optimisation in Fraud Analysis:* Balances exploration (new anomaly patterns) and exploitation (known fraud indicators) to improve detection performance.

DB-BOA thus acts as a meta-optimiser for the entire financial ecosystem, ensuring that every component — from smart contracts to model updates to economic rewards — operates under minimal delay and maximal accuracy.

4.1.4 ADTCN: Adaptive Deep Temporal Context Network

Background and Theoretical Foundation. The Adaptive Deep Temporal Context Network (ADTCN) is a temporal one-dimensional convolutional detector applied to a sliding window of recent transactions, designed to model short- and long-range dependencies in transaction sequences. In financial systems, where transactions occur continuously and exhibit evolving temporal patterns, processing an ordered window rather than a single transaction enables detection of anomalies that point-wise models cannot capture.

Architecture and Components. The detector processes a sliding window of the ten most recent transactions, each represented by 33 input features: the 30 base features (the PCA components V1–V28, the standardised Amount, and Time) plus three temporal-amount recurrence features computed during preprocessing. The window passes through two stacked one-dimensional convolution layers (kernel size 3, ReLU activations) that expand the channel dimension from F to $2F$, followed by **global max-pooling over the time axis** and a linear layer that outputs the normal/fraud logits. The base-paper conceptual blocks map onto this realisation as follows:

- *Multi-Modal Joint Embedding (MJE):* the raw multi-feature input vector for each transaction (V1–V28, Amount, Time, and the three recurrence features).
- *Temporal Context Learning (TCL):* the two stacked one-dimensional convolutions over the ordered ten-step window.
- *Multiple Time-Scale Temporal Attention (MTTA):* realised as the global max-pooling operator that selects the most salient time step (a pooling operator used in place of the base paper's attention).

The three temporal-amount recurrence features — the recurrence of similar transaction amounts before and after each transaction, and a degree ratio, computed from the discretised Amount stream — are appended to the 30 base features to form the 33-dimensional per-transaction input. Additional temporal context features (rolling means/standard deviations over windows $\{5, 10, 20\}$ and first/second differences) are also engineered during preprocessing but are not consumed by the convolutional detector, which derives its temporal context from the ten-step sliding window itself. Extreme class imbalance (0.17% fraud) is handled with a class-weighted cross-entropy loss rather than resampling, and the optimiser is Adam with learning rate 10^{-3}.

Optimisation and Objective. The convolutional filter count and steps per epoch are tuned by DB-BOA according to a bounded, MCC-dominated fitness:

$$\mathrm{Obf}_2 = \arg\max_{H_{nD},\, S_{eD}} \big[\, 2\,\mathrm{MCC} + \mathrm{Spec} + \mathrm{Pre} + \mathrm{NPV} \,\big]. \tag{4.1}$$

This rewards predictive power (MCC, precision, NPV) and low false alarms (specificity) — critical in finance. The bounded specificity term replaces the unbounded 1/FPR reward of the base paper's objective, which diverges as FPR $\to 0$ and causes every false-positive-free candidate to tie at an unbounded best score, making the search degenerate. With the corrected objective, DB-BOA's automatically selected configuration ($H^*_{nD} = 142$, $S^*_{eD} = 76$) yields a working detector without manual search; as reported in Chapter 5, it does not exceed a carefully hand-tuned default — DB-BOA's demonstrated value is automation and leader selection, not a detection-accuracy gain.

Applications and Interpretations:

- *Real-Time Fraud Detection:* Identifies fraudulent card usage or synthetic identity fraud in near real-time by analysing transaction sequences.
- *Anti-Money Laundering (AML):* Learns temporal relationships between repeated transfers and hidden intermediaries, flagging suspicious transaction cycles.
- *Credit Risk Analysis:* Detects temporal deviations in repayment behaviour across clients in microfinance or credit scoring systems.
- *Market Manipulation Detection:* Monitors blockchain-based stock trading for wash trades, spoofing, or pump-and-dump patterns by analysing multi-scale trade intervals.

By continuously learning from federated updates aggregated via the DP/Krum/Shapley federation layer, ADTCN maintains robustness across changing financial contexts, adapting to evolving fraud techniques or seasonal variations in trading behaviour.

4.1.5 Privacy-Preserving, Fairness-Aware Federated Aggregation

PRIMARY CONTRIBUTION

Each federation round combines three complementary mechanisms — differentially private weight sharing, Krum robust selection, and exact Shapley-value contribution attribution — and binds the Shapley contribution weights to an on-chain token incentive enforced by Fabric chaincode. Rather than claiming priority, this thesis *characterises* the privacy–incentive coupling of this Shapley-driven, chaincode-enforced reward mechanism on a real permissioned blockchain, extending prior Shapley-on-blockchain incentive work (FedCoin [17], SI-ChainFL [43]) with a deployed Hyperledger Fabric implementation and a noise→fidelity→reward-error analysis.

Table 4.1: Integrated Architectural Flow of the FL-ADTCN Framework

Layer	**Technology Used**	**Core Purpose and Function**
Consensus Layer	Raft Ordering in Hyperledger Fabric	Validates transactions and federated updates under authorised consortium nodes (BankA, BankB, BankC).
Optimisation Layer	DB-BOA (2 roles)	Tunes ADTCN hyperparameters and selects optimal leader nodes.
Federation Layer	DP + Krum + Shapley	Differentially private weight sharing, Krum robust selection, and exact Shapley contribution attribution; Shapley weights drive the token split.
Learning Layer	Federated ADTCN	Performs decentralised temporal fraud detection across the three-bank consortium.
Smart Contract Layer	DBBOAContract (Node.js Chaincode)	Executes transaction validation, token reward allocation, leader election recording, and federation round logging.
Ledger Layer	Fabric World State and Blockchain Ledger	Stores immutable records of financial transactions, model metrics, federation weights, token balances, and validation outcomes.

(1) Differentially private weight sharing. Before any sharing, each organisation's weight tensors are L_2-clipped to a norm bound C and perturbed by Gaussian noise $\mathcal{N}(0, \sigma^2)$ with $\sigma = C\sqrt{2\ln(1.25/\delta)}/\epsilon$ (Dwork et al. [2]). At the deployment budget $\epsilon = 1.0$, $\delta = 10^{-5}$, $C = 1$ this gives $\sigma \approx 4.84$ and an (ϵ, δ) guarantee. As discussed in Chapter 5, $\epsilon = 1.0$ is a deliberately tight budget at which the weight noise exceeds the signal; the deployment therefore disables weight-channel DP and instead carries privacy on the incentive (output) channel — the configuration the central characterisation shows to be both private and faithful.

(2) Krum robust selection. Each organisation's update is scored by the sum of squared L_2 distances to its nearest neighbours; the lowest-scoring (most consensus-aligned) update is chosen as the global model (Blanchard et al. [4]). In the default three-organisation deployment this reduces to outlier rejection (the largest f admissible under $n \geq 2f+3$ is 0). Statistical Byzantine tolerance for $f \geq 1$ is demonstrated separately in the $n = 5, f = 1$ and $n = 7, f = 2$ regimes against weight-level poisoning (Chapter 5); a colluding *majority* ($> f$) is handled by the complementary economic mechanism.

(3) Exact Shapley contribution attribution. Each organisation i is assigned its Shapley value $\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!\,(n-|S|-1)!}{n!}\,[v(S \cup \{i\}) - v(S)]$, where the coalition value $v(S)$ is the balanced accuracy of the equally-averaged model of the organisations in S on a shared validation set (Wang et al. [12]). For $n = 3$ this evaluates all seven non-empty coalitions exactly. Negative values are clipped and the weights normalised to sum to one. Balanced accuracy is used because it is bounded and robust to the 0.17% class imbalance, so a participant that catches fraud without over-flagging scores near one while an always-fraud adversary scores near one half and lowers any coalition it joins.

Economic coupling. The Shapley weights $\mathbf{w}^*$ are written to the Fabric ledger, and `recordFederationRound` distributes the 20-token federation pool as $\text{tokens}_i = \lfloor 20\, w_i^* \rfloor$. Because reward is the direct output of the Shapley analysis, an organisation that contributes a stronger model earns more automatically, and one that degrades the global model earns essentially nothing — without human adjudication. On the single-source deployment data the three banks are stratified volume-splits of the same dataset, so their marginal contributions are near-identical and the split is near-uniform (≈ 0.333 each); the mechanism's value where contributions genuinely differ is established by the privacy–incentive characterisation in Chapter 5.

4.1.6 Objectives, Constraints, and Performance Goals

Performance objectives. On the consensus path, DB-BOA-driven leader selection aims to reduce simulated block latency relative to random selection. On the learning side, the federated ADTCN aims for robust fraud-detection performance on the extremely imbalanced ULB data, measured by MCC and balanced accuracy rather than raw accuracy (which is uninformative at 0.17% fraud). Reported throughput is computed by the consensus *simulation* layer from node resource scores; the per-round latency was measured on a single-host, two-organisation Fabric deployment (mean ≈ 2.1 s), not on a live multi-host network. Both characterise relative behaviour rather than production performance (see Limitations).

Security objectives. Data privacy is enforced on two fronts: a federated learning protocol in which raw transaction records never leave institutional boundaries, and differential privacy applied to the shared statistics. Smart-contract integrity is maintained through immutable Raft-backed records of model updates, contribution weights, and incentive events, enabling non-repudiable audit trails. Byzantine resilience is provided by two complementary layers: Krum robust selection at the parameter level (in the $n \geq 2f+3$ regime) and an economic mechanism that isolates a colluding majority through the Shapley-weighted reward and reputation loop.

System constraints. Scalability is intentionally scoped to a consortium environment in which validator roles are restricted to accredited financial entities (BankA, BankB, BankC). The learning pipeline is trained on the Kaggle ULB Credit Card Fraud Detection dataset; subsequent phases will extend to real-time blockchain-native financial data streams to capture production-scale heterogeneity and temporal dynamics.

Improvement targets. On the consensus path, the objective is to reduce simulated block latency relative to random leader selection through DB-BOA-driven leader scheduling. On the detection side, the goal is a working federated detector obtained without manual tuning; the verified detector reaches MCC 0.677 at the weight-DP-off deployment operating point (Chapter 5). Note that this does not surpass a carefully hand-tuned default — DB-BOA's demonstrated contribution is automation and leader selection, not a detection-accuracy gain (reported honestly in Chapter 5).

4.1.7 Methodology Process Flow

The research follows a structured six-phase pipeline. Each phase builds on the previous, culminating in a fully integrated, incentivised federated fraud detection system.

1. **Phase 1 — Leader Selection (DB-BOA):** DB-BOA reads node resource metrics (CT, CC, MS) from the Fabric ledger with reputation discounts applied, and selects the optimal leader node for each consensus round. Results written on-chain.

2. **Phase 2 — Hyperparameter Optimisation (DB-BOA):** DB-BOA searches the two-dimensional space (H_{nD}, S_{eD}) to maximise the bounded Obf_2 on a validation subsample (epoch count fixed). Optimal parameters stored on-chain under `HYPERPARAMS`. Runs once at system startup.

3. **Phase 3 — ADTCN Local Training:** Each institution trains its local ADTCN model using DB-BOA-optimised hyperparameters on its private data shard. Fraud detection verdicts submitted to the blockchain; chaincode applies token incentives.

4. **Phase 4 — Federated Aggregation (DP + Krum + Shapley):** Every five consensus rounds, the federation manager applies differentially private weight sharing, Krum robust selection, and exact Shapley contribution attribution. The global model is computed and distributed to all organisations; token rewards are distributed proportionally to the Shapley weights.

5. **Phase 5 — Blockchain Integration (Hyperledger Fabric):** All phases execute against the live Hyperledger Fabric test-network. The `DBBOAContract` chaincode enforces incentive rules, records all decisions immutably, and emits events to the monitoring dashboard via the Node.js API server.

6. **Phase 6 — Evaluation and Adversarial Testing:** The integrated system is evaluated under normal operation (five simulated consensus rounds and three federation rounds) and under a 15-round Byzantine attack (BankC configured as a constant-fraud reporter) to validate incentive mechanism resilience.

4.2 Preliminary Design or Design (Model) Specification

4.2.1 Dynamic Butterfly–Billiards Optimization Algorithm (DB-BOA): Origin, Design, and Functional Role

Introduction. The Dynamic Butterfly–Billiards Optimization Algorithm (DB-BOA) forms the optimisation backbone of the proposed blockchain–learning architecture. It operates as the adaptive intelligence that continuously tunes the blockchain's consensus parameters and the deep-learning model's hyperparameters to maintain both computational efficiency and predictive accuracy. Unlike traditional optimisation methods with fixed mathematical gradients, DB-BOA is a hybrid

meta-heuristic algorithm — it learns through simulation and stochastic adaptation, not calculus. Its design philosophy combines the broad exploration capacity of biological swarm intelligence with the fine precision of physical motion dynamics. The outcome is an optimiser capable of balancing speed, stability, and scalability within a decentralised network.

Evolution and Theoretical Lineage. DB-BOA evolved through the fusion of two independent optimisation paradigms:

- *Dynamic Butterfly Optimization Algorithm (DBOA)* — inspired by the olfactory behaviour of butterflies. Each candidate solution acts like a butterfly that senses "fragrance intensity" proportional to fitness, guiding it toward better regions of the search space.
  Strengths: excellent at global exploration and avoiding local minima.
  Weaknesses: higher computational time and slower convergence on large datasets.

- *Billiards Optimization Algorithm (BOA)* — derived from the physics of elastic collisions among billiard balls. Each ball represents a solution, and pockets represent optimal targets.
  Strengths: extremely effective for local exploitation and quick convergence.
  Weaknesses: prone to premature convergence when diversity in solutions is lost.

The hybrid DB-BOA unites both ideas to exploit their strengths and overcome their individual shortcomings. It alternates between DBOA-driven exploration and BOA-driven refinement during each iteration. This dual mechanism allows the optimiser to search globally during early iterations and converge rapidly as it approaches the optimal solution.

Table 4.2: Comparison of DBOA, BOA, and DB-BOA

Feature	**DBOA**	**BOA**	**DB-BOA**
Inspiration	Butterfly foraging (biological)	Billiard physics (mechanical)	Hybrid biological–physical
Focus	Exploration	Exploitation	Adaptive balance
Convergence	Slower, global	Faster, local	Balanced
Limitation	Time-intensive	Local traps	Adaptive switch probability
Application in this work	Broad parameter search	Fine leader selection	Combined: hyperparameter tuning + leader selection

Mathematical Structure. The DB-BOA workflow begins with a randomly initialised population of candidate solutions

$$Y = \{y_1, y_2, \ldots, y_n\}. \tag{4.2}$$

For each iteration t, a decision rule determines which parent algorithm governs the

update:

$$Y_{t+1} = \begin{cases} \text{DBOA}(Y_t), & \text{if rand} < \tau_t, \\ \text{BOA}(Y_t), & \text{otherwise,} \end{cases} \qquad \tau_t = 1 - \frac{\left|\text{bestfit}^{(t)} - \text{worstfit}^{(t)}\right|}{\max\left(\left|\text{bestfit}^{(t)}\right|,\, \left|\text{worstfit}^{(t)}\right|,\, \varepsilon_0\right)}, \tag{4.3}$$

where $\text{bestfit}^{(t)}$ and $\text{worstfit}^{(t)}$ are the best and worst objective values in the population at iteration t and ε_0 is a small constant guarding against division by zero. The threshold τ_t is a range-normalised convergence measure — the implemented form of the base paper's bestfit/worstfit ratio criterion, valid for both positive costs and negated (maximisation) objectives: when the population has converged (best $\approx$ worst) $\tau_t \to 1$ and the DBOA exploitation step dominates; when the population is diverse τ_t falls and the BOA exploration step dominates. This mechanism is fully adaptive without requiring manual scheduling of the switch probability.
When the BOA phase is selected, each candidate updates its position according to:

$$y_{j,e}^{\text{new}} = y_{j,e} + s\big(Q_{j,e} - J\, y_{j,e}\big), \tag{4.4}$$

where $s \in [0, 1]$ is the step size controlling exploration intensity, $Q_{j,e}$ is the target "pocket" (a better solution), and J is a control coefficient regulating rebound strength.
When DBOA is selected, its butterfly search moves are followed by the Local Search Algorithm based on Mutation (LSAM), which perturbs the current best solution y^* with range-scaled Gaussian mutation:

$$y_{\text{new}}^* = y^* + \mu\, \mathcal{N}(0, I) \odot (\mathbf{ub} - \mathbf{lb}), \tag{4.5}$$

with mutation scale $\mu = 0.1$ over five LSAM iterations per call; if the mutant is not fitter, a randomly chosen population member is mutated instead. (The base paper describes this diversity-restoring step with Lévy-flight perturbations; the implementation realises it with the bounded Gaussian mutation shown, which serves the same purpose.) This mutation restores population diversity around the current optimum, preventing premature convergence.

4.2.2 Objective Functions and Optimisation Goals

(a) Blockchain-Level Optimisation (DB-BOA — Leader Selection). DB-BOA minimises the combined resource cost of blockchain consensus:

$$\text{Obf}_1 = \arg\min_{L_{\text{bbc}}} \big(CT + CC + MS\big), \tag{4.6}$$

where:

- **CT (Computation Time):** average CPU time to validate and commit a block,
- **CC (Communication Cost):** network message load during endorsement and ordering,
- **MS (Memory Size):** runtime ledger memory footprint per block.

This objective promotes the selection of a leader node that yields the lowest operational overhead while sustaining throughput and reliability.

(b) Model-Level Optimisation (DB-BOA — Hyperparameter Tuning). Simultaneously, DB-BOA maximises a bounded, MCC-dominated learning fitness for the ADTCN model:

$$\mathrm{Obf}_2 = \arg\max_{H_{nD},\, S_{eD}} \big[\, 2\,\mathrm{MCC} + \mathrm{Spec} + \mathrm{Pre} + \mathrm{NPV} \,\big], \tag{4.7}$$

where MCC, Spec (specificity), Pre, and NPV are standard classification metrics. The bounded specificity term replaces the base paper's unbounded 1/FPR reward, which diverges as FPR $\to 0$ and ties every false-positive-free candidate at an unbounded best score, making the search degenerate. The MCC weight ensures the objective tracks balanced quality on the extremely imbalanced data. Only two dimensions are searched (H_{nD}, S_{eD}); the epoch count is fixed.

(c) Federated Aggregation (handled by the federation layer, not DB-BOA). The federated aggregation weights are produced by the dedicated DP/Krum/Shapley federation layer of Section 4.1.5, not by DB-BOA. The global model is $\theta_{\mathrm{global}}(\mathbf{w}) = \sum_{i=1}^{K} w_i \theta_i$ with $\mathbf{w}$ the normalised exact Shapley contribution weights, evaluated on a shared validation set. DB-BOA therefore serves the infrastructural (consensus) and analytical (learning) subsystems, while the economic (incentive) subsystem is driven by the Shapley signal.

4.2.3 Role in Blockchain Leader Selection with Reputation Discounting

Within the Hyperledger Fabric consortium network, DB-BOA functions as the leader-selection controller. Each peer node periodically provides three resource metrics read from the Fabric ledger:

- **CT (Computation Time):** time to validate and propose a block,
- **CC (Communication Cost):** total message volume during endorsement,
- **MS (Memory Size):** ledger memory footprint per block.

A reputation discount factor $\rho_i \in [0.5, 2.0]$ is applied, making the effective objective for peer i:

$$\mathrm{Obf}_1^{(i)} = \frac{CT_i + CC_i + MS_i}{\rho_i}. \tag{4.8}$$

Peers with higher reputation have a lower effective cost, making them more competitive in leader selection. Reputation updates follow:

$$\rho_i^{(r+1)} = \begin{cases} \min\!\left(\rho_i^{(r)} + 0.02,\ 2.0\right) & \text{if round } r \text{ succeeds,} \\ \max\!\left(\rho_i^{(r)} - 0.05,\ 0.5\right) & \text{if round } r \text{ fails.} \end{cases} \tag{4.9}$$

This creates a multi-round feedback loop: reliably performing nodes are progressively preferred for leadership, earn more leadership rewards, and maintain higher reputation — a self-reinforcing cycle of good behaviour.
Leader selection runs over a ten-node simulated consortium, where DB-BOA minimises the reputation-discounted resource cost each round. The selection frequencies reported by this layer are computed from node resource scores in simulation

(`leader_block.py`); the per-round consensus latency is taken from a real single-host, two-organisation Fabric measurement (mean $\approx$ 2.1 s), not a live multi-host network. They characterise the relative behaviour of reputation-aware selection rather than production throughput, and are revisited in the Limitations.

4.2.4 Exact Shapley Contribution Attribution — Detailed Design

Coalition Evaluation. For the $n = 3$ consortium, the federation manager evaluates all $2^3 - 1 = 7$ non-empty coalitions exactly. The value $v(S)$ of a coalition S is the balanced accuracy of the equally-averaged model of the organisations in S on a shared validation set. The Shapley value ϕ_i for organisation i is its average marginal contribution across all coalitions; negative values are clipped and the resulting weights normalised to sum to one. Balanced accuracy is used as the coalition value because it is bounded and robust to the 0.17% class imbalance.

Why This Improves on FedAvg. FedAvg assigns $w_i = n_i / \sum n_j$ (sample-count weighting), which *assumes* contribution tracks data volume. Exact Shapley instead *measures* each organisation's marginal contribution to the collaborative model, the unique attribution satisfying the efficiency, symmetry, and null-player axioms. On genuinely identical contributors it correctly returns a near-equal split; where contributions differ it rewards the stronger model and assigns a poisoning participant near-zero weight.

Incentive Token Distribution. Once $\mathbf{w}^*$ is written to the Fabric ledger, the `recordFederationRound` chaincode function distributes the 20-token federation pool as $\text{tokens}_i = \lfloor 20 \cdot w_i^* \rfloor$. On the single-source deployment data the per-round split is near-uniform (≈ 0.333 each, ≈ 6.67 tokens), because the three banks are stratified volume-splits of the same dataset; under the Byzantine attack (BankC always-fraud) the Shapley layer assigns the poisoned organisation a weight of 0.000 and the incentive contract drains its token balance and floors its reputation, as reported in Chapter 5.

Table 4.3: Complete token incentive structure enforced by DBBOAContract chaincode

Event	**Tokens**	**Enforced by Chaincode Function**
Fraud verdict confirmed by consensus majority	+10	`recordFraudResult`
Confirmed verdict AND latency $<$ 300 ms	+15	`recordConsensusRound`
Selected as leader AND consensus round succeeds	+10	`recordConsensusRound`
Verdict disputed by consensus majority	−2	`recordFraudResult`
Consensus round fails under current leader	−2	`recordConsensusRound`
Federation participation (20-token pool, split by Shapley weight)	$+\lfloor 20 w_i^* \rfloor$	`recordFederationRound`

4.2.5 Practical Advantages and Observed Behaviour

- **Automation:** DB-BOA selects a working ADTCN configuration without manual search. The bounded objective surface is flat near a well-chosen configuration, so the optimiser converges quickly; its demonstrated value is automation, not a detection-accuracy gain over a hand-tune (reported honestly in Chapter 5).
- **Efficiency:** DB-BOA reputation-aware leader selection reduces simulated consensus latency relative to random selection.
- **Adaptability:** Dynamic switching maintains optimiser performance across diverse data distributions (stratified and volume-skewed consortium partitions).
- **Economic Alignment:** Exact Shapley contribution weights determine both model-quality contribution and economic reward — a measured, axiomatic attribution that replaces FedAvg's assumed equal weighting, extending the Shapley-on-blockchain incentive line [17] with a deployed Fabric implementation.
- **Byzantine Resilience:** Krum rejects weight-level poisoning in the $n \geq 2f+3$ regime, and the economic mechanism isolates a colluding majority through the Shapley-weighted reward and reputation loop.

4.2.6 Current Constraints and Future Development

Table 4.4: Current implementation constraints and planned development roadmap for FL-ADTCN

Type	**Description**	**Impact**	**Planned Enhancement**
Consortium scale	Prototype validated with 3 organisations. Real consortia may involve 10–50 banks.	Performance at scale unvalidated.	Extend Fabric channel and DB-BOA to $K > 3$ organisations.
Resource metrics	Node CT, CC, MS simulated from realistic distributions rather than collected from real hardware monitoring.	Metrics may not reflect production variance.	Deploy Prometheus-based peer monitoring agents.
Shared validation set	Exact Shapley attribution requires a shared anonymised validation set agreed by all consortium members.	Coordination overhead not resolved in prototype.	Design consortium-negotiated validation set construction protocol.
Adaptive adversaries	Krum tested for weight poisoning ($n \geq 2f+3$); economic isolation tested against a colluding majority.	Sophisticated adaptive poisoning near the honest cluster only partially mitigated.	Add FLTrust or coordinate-wise median; multi-seed adaptive-attacker study.
Dataset scope	Kaggle ULB dataset covers European credit card transactions over 48 hours.	Generalisation to other geographies and time horizons requires validation.	Extend to real-time streaming transaction logs.

DB-BOA represents a pivotal step in merging optimisation intelligence with permissioned blockchain governance. Within this research, DB-BOA is deployed in *two live roles* under a single adaptive loop: deep-learning hyperparameter search for the ADTCN detector, and consensus leader selection. A third role — direct optimisation of per-organisation federation weights — is implemented but disabled by default; fair contribution attribution and robust aggregation are instead handled by the differential-privacy / Krum / Shapley aggregation layer described in the following sections. The current stage validates these roles on a real Hyperledger Fabric

test-network, confirming that hybrid optimisation can effectively bridge blockchain consensus, machine-learning intelligence, and economic incentive alignment.

4.2.7 Adaptive Deep Temporal Context Network (ADTCN): Architecture and Design

The Adaptive Deep Temporal Context Network (ADTCN) serves as the intelligence layer of the hybrid blockchain–learning framework, enabling temporal anomaly detection in consortium-based financial systems. Within this architecture, ADTCN functions as the decision brain — processing ordered windows of recent transactions and producing fraud probability scores. Its structure combines a multi-feature input, stacked one-dimensional convolutions over the window, and global pooling over time, with the filter count and steps-per-epoch tuned by DB-BOA.

Theoretical Foundation and Evolution. ADTCN extends the Deep Temporal Context Network (DTCN) paradigm for temporal sequence tasks. Conventional DTCN suffered from fixed time-window rigidity and a lack of adaptability to heterogeneous data. The Adaptive DTCN introduces the following practical advances:

Table 4.5: Evolution from DTCN to ADTCN across modelling, aggregation, data integration, and optimisation

Feature	**DTCN**	**ADTCN (this work)**
Temporal Modeling	Fixed periodic window	Sliding ten-step window over recent transactions
Temporal Aggregation	Single-scale pooling	Global max-pooling over the time axis
Data Integration	Static input streams	Federated + blockchain feedback adaptation
Optimization	Manual tuning	DB-BOA hyperparameter optimisation (filters, steps)

4.2.8 ADTCN Architectural Design

The ADTCN architecture is composed of the following modules, with the base-paper conceptual block names mapped onto their actual realisation [40]:

- **(a) Multi-Modal Joint Embedding (MJE).** The raw multi-feature input vector for each transaction — the 33 input features (V1–V28 PCA components, standardised Amount, Time, and three temporal-amount recurrence features). No separate learned embedding layer is used.
- **(b) Temporal Context Learning (TCL).** Two stacked one-dimensional convolution layers (kernel size 3, padding 1, ReLU) applied over the ordered ten-step window, expanding the channel dimension from F to $2F$. The window itself supplies the temporal context.
- **(c) Multiple Time-Scale Temporal Attention (MTTA).** Realised as global max-pooling over the time axis, which selects the most salient time step — a pooling operator used in place of the base paper's attention.

4.2.9 Mathematical Formulation

Let $X = \{x_1, x_2, \ldots, x_T\}$ ($T = 10$) be the windowed input of recent transactions, each $x_t \in \mathbb{R}^{33}$. ADTCN performs three stages:
Convolutional Encoding:

$$H = \mathrm{ReLU}\big(\mathrm{Conv1d}_{F\to 2F}\big(\mathrm{ReLU}(\mathrm{Conv1d}_{33\to F}(X))\big)\big), \tag{4.10}$$

two stacked 1-D convolutions over the time axis (kernel size 3) producing channel maps $H \in \mathbb{R}^{2F\times T}$.
Temporal Pooling:

$$c = \max_{t=1,\ldots,T} H_{:,t}, \tag{4.11}$$

global max-pooling over time, yielding the context vector $c \in \mathbb{R}^{2F}$ (the salient-timestep summary that stands in for attention).
Classification:

$$\hat{y} = \mathrm{Linear}_{2F\to 2}(c), \tag{4.12}$$

two logits for the normal/fraud classes, trained with class-weighted cross-entropy.
Adaptive Objective (DB-BOA tuned):

$$\mathrm{Obf}_2 = \arg\max_{H_{nD},\, S_{eD}} \big[\, 2\,\mathrm{MCC} + \mathrm{Spec} + \mathrm{Pre} + \mathrm{NPV}\,\big], \tag{4.13}$$

where H_{nD} (filter count) and S_{eD} (steps per epoch) are searched (the epoch count is fixed); DB-BOA's selected configuration is $H^*_{nD} = 142$, $S^*_{eD} = 76$.

4.2.10 Integration with DB-BOA and Blockchain Consensus

In the Hyperledger Fabric consortium environment, DB-BOA dynamically tunes the ADTCN parameters during runtime. The integration follows a coupled feedback cycle:

1. DB-BOA selects the optimal leader node; the leader proposes the next block containing anonymised transaction features.

2. ADTCN receives validated transaction streams from the selected leader and produces fraud probability scores.

3. Fraud verdicts are submitted to the Fabric ledger; the chaincode applies token incentives based on consensus agreement.

4. Every 5 rounds, the federation manager aggregates local ADTCN models into a global model via the DP/Krum/Shapley layer; the global model is redistributed to all institutions.

5. DB-BOA re-reads current node metrics at every consensus round for leader selection; the hyperparameter search can be re-run if sustained performance degradation is observed (a manual operation in the current implementation).

This coupling allows adaptive control of the consensus layer — if a node's effective cost rises, DB-BOA re-selects the leader at the next round; re-tuning the detector remains a manual operation.

At the weight-DP-off deployment operating point, the DB-BOA-tuned ADTCN reaches MCC 0.677 on the held-out ULB test set (Chapter 5). These are honest numbers for a lightweight one-dimensional CNN on raw PCA features; they are not the base paper's 95.45% nor an inflated 99.31% — those are not produced by this implementation.

Practical Advantages and Behavioral Insights.

- **Resilience:** Handles high-frequency transaction bursts without degrading accuracy.
- **Adaptability:** Learns temporal patterns over the ten-step sliding window via stacked 1-D convolutions.
- **Robustness:** Maintains stability under data heterogeneity through stratified partitioning that preserves the global fraud rate in each shard.
- **Scalability:** The contribution-attribution layer remains tractable as the federation grows via Monte-Carlo Shapley (Chapter 5).

4.2.11 Federated Adaptive Deep Temporal Context Network (FL-ADTCN): Distributed Design and Federation Modes

Introduction

The Federated Adaptive Deep Temporal Context Network (FL-ADTCN) is the distributed and privacy-preserving extension of the ADTCN model, designed to detect financial anomalies and fraudulent activity across consortium blockchain environments. It enables collaborative intelligence among the three participating banking organisations (BankA, BankB, BankC) without exchanging raw transaction data. Unlike centralised deep learning, FL-ADTCN employs the Federated Learning paradigm where each node locally trains a private model on its own dataset and only shares model parameters (not data) with the global aggregator — implemented here on Hyperledger Fabric via the DBBOAContract chaincode.
Through integration with DB-BOA (hyperparameter tuning and consensus leader selection) and the DP/Krum/Shapley federation layer, FL-ADTCN achieves automated detector configuration, optimised consensus leadership, and measured contribution weights that drive the on-chain incentive, while maintaining transaction-level anomaly sensitivity.

Federated Learning Modes

FL-ADTCN was evaluated under three collaborative learning modes:

(a) Centralized Training (Baseline). All training data are aggregated into a single global model. *Advantages:* maximum accuracy and fast convergence. *Disadvantages:* high privacy risk and regulatory non-compliance.

(b) Federated IID Training. Each institution holds Independent and Identically Distributed subsets of the data — each bank's partition maintains the global 0.17% fraud rate. Ensures stable training close to the centralised baseline while preserving data sovereignty.

(c) Federated non-IID Training. Each institution holds Non-Independent and Non-Identically Distributed data — reflecting heterogeneity in institutional size (50/30/20 split across BankA, BankB, BankC). Introduces realistic imbalance, addressed through stratified partitioning that preserves the global 0.17% fraud rate in each shard.

Model Architecture Overview

The FL-ADTCN uses the same one-dimensional convolutional detector as ADTCN but operates in a distributed parameter-synchronisation framework, with the DP/Krum/Shapley federation layer replacing naive FedAvg aggregation.

Table 4.6: Layered FL-ADTCN architecture (1-D CNN realisation of the MJE–TCL–MTTA blocks)

Module	Description	Purpose
MJE (Multi-Modal Joint Embedding)	Raw 33-feature transaction input (V1–V28, Amount, Time, three recurrence features); no separate learned embedding.	Supplies the multi-feature input for each institution.
TCL (Temporal Context Learning)	Two stacked `Conv1d` layers (kernel 3, ReLU) over the ten-step window, $F \to 2F$ channels.	Learns time-aware transaction evolution over the window.
MTTA (Multi-Timescale Temporal Attention)	Global max-pooling over the time axis; selects the salient time step.	Summarises the window for the linear classifier.

Figure 4.2 shows the complete forward path of the detector as implemented, annotated with the tensor shapes at each stage.

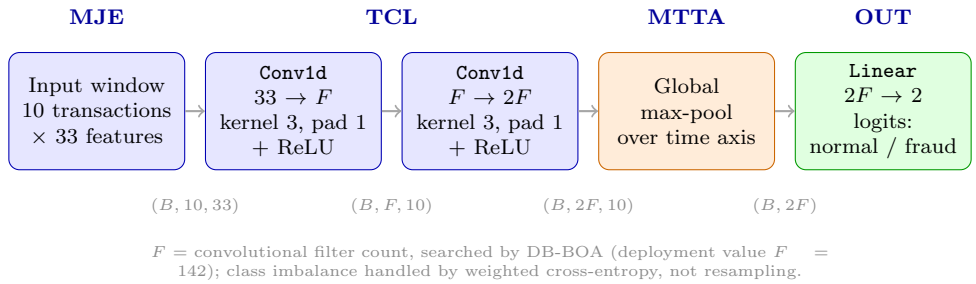


Figure 4.2: Forward path of the ADTCN detector as implemented: two stacked `Conv1d` layers over the ten-transaction window, global max-pooling over the time axis, and a linear classification head. Group labels map each stage to the MJE/TCL/MTTA blocks of Table 4.6.

The experimental configuration, the performance-metric definitions, and the quantitative results of this federated evaluation are presented in Chapter 5.

Discussion and Constraints

(a) Achievements. The framework implements, on a real Hyperledger Fabric consortium, a DP + Krum + exact-Shapley federated aggregation layer whose Shapley contribution weights drive an on-chain, chaincode-enforced token incentive; it extends the Shapley-on-blockchain incentive line [17], [43] with a deployed implementation and, as its central contribution, a characterisation of the privacy–incentive coupling of that mechanism (Chapter 5).

(b) Remaining Limitations.

Table 4.7: Key limitations in the current FL-ADTCN implementation

Category	**Description**	**Impact**
Consortium scale	Validated with 3 banks; real consortia involve more participants.	Exact Shapley is $O(2^n)$; Monte-Carlo attribution extends it but fidelity at fixed budget holds only to $n \approx 5$ (Chapter 5).
Validation set construction	Shared anonymised validation set requires consortium agreement.	Coordination overhead in production deployment.
Adaptive adversaries	Krum tested for weight poisoning ($n \geq 2f + 3$); economic isolation tested against a colluding majority.	Subtle poisoning near the honest cluster only partially mitigated.
Simulated resource metrics	CT, CC, MS collected from simulations rather than real peer monitoring.	May not fully reflect production hardware variance.
Dataset generality	Kaggle ULB covers European transactions over 48 hours only.	Temporal and geographic generalisation unvalidated.

Table 4.8: Future enhancement roadmap for FL-ADTCN

Challenge	Proposed Solution
Non-IID client drift	Stratified partitioning preserves the global fraud rate per shard; future: proximal methods (FedProx), FedNova, or FedAdam.
Consortium scalability	Extend Fabric channel to $K > 3$ orgs; use Monte-Carlo Shapley with sample budget scaling as $O(n \log n)$.
Byzantine-robust aggregation	Krum already integrated; future: coordinate-wise median or FLTrust for adaptive attackers.
Differential privacy	Relax the budget with DP-SGD at the gradient level; weight-channel DP is disabled at deployment in favour of incentive-channel DP.
Quantum-resistant identity	Replace X.509 with lattice-based or hash-based certificate primitives.
Real-time streaming	Extend ADTCN to process live transaction streams at production volumes.

4.3 Data Collection

Data collection for this project draws from both the conceptual framework of the base paper and practical experimentation.
In the base paper, customer financial data (transactions and records from stock market exchanges) is collected and secured directly on the Private Ethereum Consortium Blockchain (PEC-BC). No specific public dataset is used; data is described generically as "customer financial data" and simulated in MATLAB 2020a for performance evaluation (e.g., varying block sizes 5,000–14,000, transaction rates 100–400 tx/sec, noise 5–25%). The dataset is not publicly available due to privacy concerns.
For experimental validation of the ADTCN component and the full FL-ADTCN consortium, the **Kaggle ULB Credit Card Fraud Detection dataset** was employed. This publicly available dataset contains 284,807 anonymised European credit card transactions over 48 hours, with 31 features (Time, Amount, V1–V28 PCA-transformed, Class label). Fraud cases: 492 (0.17%), normal: 284,315 (99.83%). Data was collected from real financial logs, de-identified (PII removed), and PCA-transformed by the original providers. The dataset was downloaded directly from Kaggle and used without modification to source.

Development Environment

The full FL-ADTCN system was implemented and validated on the following infrastructure:

Table 4.9: Development environment specification for the FL-ADTCN implementation

Component	**Specification**
Operating System	Ubuntu 22.04 LTS
Python	3.10.12 (ML and optimisation layer)
Node.js	18.17.0 LTS (API server and DBBOAContract chaincode)
Docker	24.0.5 (Hyperledger Fabric network containers)
Hyperledger Fabric	v2.5.4 (latest LTS release)
Fabric SDK	`fabric-network` 2.2.20 (wallet-based Node.js SDK)
ML Framework	PyTorch (CPU)
State Database	CouchDB 3.3.3 (Fabric world state, rich JSON queries)
Optimiser	Adam, learning rate 10^{-3}, class-weighted cross-entropy loss

Consortium Data Distribution

For federated training, the Kaggle dataset was partitioned across the three banking organisations using stratified sampling to maintain the global 0.17% fraud rate in each partition:

Table 4.10: Stratified dataset partitioning across the three-bank consortium

Organisation	**Samples**	**Fraud Count**	**Fraud Rate**	**Split**
BankA (Org1MSP)	142,403	246	0.17%	50%
BankB (Org2MSP)	85,442	148	0.17%	30%
BankC (Org3MSP)	56,962	98	0.17%	20%
Total	**284,807**	**492**	**0.17%**	**100%**

Stratified sampling ensures all three partitions maintain the global fraud rate identically, eliminating the extreme label skew that causes non-IID gradient conflicts under raw partitioning. Figure 4.3 visualises both the severity of the global class imbalance and the per-bank shard composition of Table 4.10.

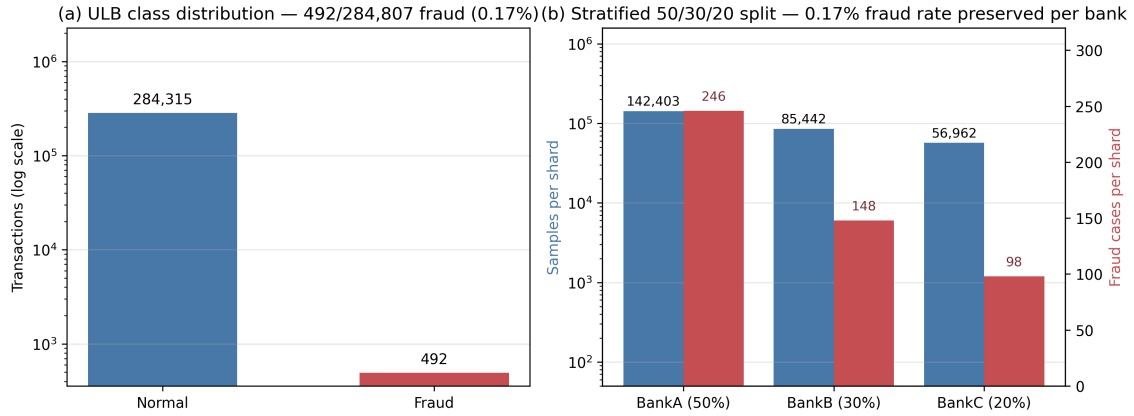


Figure 4.3: The ULB dataset and its consortium partitioning, computed from the raw `creditcard.csv`. (a) Global class distribution: 492 fraud cases among 284,807 transactions (0.17%), shown on a log scale. (b) Stratified 50/30/20 split across the three banks; each shard preserves the 0.17% fraud rate (246/148/98 fraud cases respectively).

Hyperledger Fabric Network Deployment

The on-chain measurement network was deployed using the `fabric-samples` test-network as the base, with CouchDB state databases and the `DBBOAContract` chaincode packaged as a Chaincode-as-a-Service (CCaaS). Consistent with the scope statement in Chapter 5, the physical deployment instantiates **two peer organisations on a single host** — the configuration on which all on-chain measurements were taken — while the three-bank consortium of the federated experiments (BankA, BankB, BankC) runs in the off-chain simulation layer, with its per-node state (tokens, reputation, resource metrics) recorded in the chaincode's world state. The deployment sequence was fully automated via shell scripts:

1. **Certificate Authority setup:** Each organisation's CA (Fabric CA v1.5) issued X.509 certificates to its administrator and peer identities; a separate CA serves the orderer organisation.

2. **Container startup:** The peer nodes (`peer0.org1`, `peer0.org2`), their CouchDB state databases, the CA containers, and the Raft orderer were launched via Docker Compose.

3. **Channel creation:** The `mychannel` channel was created and joined by both organisations under the test-network's default majority endorsement policy.

4. **Chaincode deployment:** `DBBOAContract` was packaged as a Node.js CCaaS chaincode, installed on both peers, approved by both organisations, and committed to the channel.

5. **API server startup:** The Node.js API server connected to `peer0.org1` via gRPC and enrolled organisation administrator identities in local filesystem wallets using the `fabric-network` 2.2.20 SDK.

Table 4.11 lists the deployed containers; the listing matches the live capture in Figure 5.1. The three-organisation consortium topology of the design (Section 4.6) remains the deployment target for a production rollout.

Table 4.11: Hyperledger Fabric Docker container deployment summary (single-host, two-organisation measurement network; cf. Figure 5.1).

Container	**Role**	**Port**
`peer0.org1.example.com`	Org1 peer node	7051
`peer0.org2.example.com`	Org2 peer node	9051
`couchdb0`, `couchdb1`	CouchDB world-state databases	internal
`ca_org1`	Org1 Certificate Authority	7054
`ca_org2`	Org2 Certificate Authority	8054
`ca_orderer`	Orderer Certificate Authority	9054
`orderer.example.com`	Raft ordering service	7050
`peer0org1_db-boa_ccaas`	`DBBOAContract` chaincode-as-a-service (Org1 peer)	9999
`peer0org2_db-boa_ccaas`	`DBBOAContract` chaincode-as-a-service (Org2 peer)	9999

Non-IID Data Heterogeneity Solutions

Two techniques address the non-IID challenge arising from unequal data distribution across the three banks:

1. **Stratified Partitioning:** Each bank's data partition maintains the global 0.17% fraud rate, eliminating the extreme label skew that causes contradictory gradient directions across clients and drives raw non-IID FedAvg toward a degenerate model.

2. **Shapley Contribution Weighting:** The federation layer assigns each institution its exact Shapley contribution weight on a shared validation set, replacing FedAvg's assumed equal weighting with a measured one; this rewards institutions whose models genuinely improve the global model and assigns near-zero weight to a poisoning participant.

Proximal methods such as FedProx (a $\frac{\mu}{2}\|\theta_i - \theta_{\text{global}}\|^2$ regulariser on local training) are a natural extension for more severely heterogeneous deployments but are *not* implemented in the current pipeline.

4.3.1 Data Cleaning

Missing Values: No missing values were present in the Kaggle dataset (0% NaN across all features and Class label after loading). The Class column was coerced to numeric (int64) and validated; invalid entries (if any) were dropped, but none occurred.
Outliers: No outliers were removed. The Amount feature shows high variance (min 0.00, max 25,691.16, median 22.00), but extremes are preserved because fraud can

occur at any value (e.g., small "testing" charges or large legitimate transfers). Temporal outliers (unusual timestamps) were retained for sequence context in ADTCN. PCA features (V1–V28) are already bounded and consistent.
Inconsistencies: Temporal ordering was verified (timestamps increasing). No duplicates, negative amounts, or invalid Class values (only 0/1). All features numeric and consistent. Result: No records removed (0% data loss). Data quality remained excellent.

Table 4.12: Statistical Summary of Key Features

Feature	**Mean**	**Std Dev**	**Min**	**25%**	**50%**	**75%**	**Max**
Amount	88.35	250.12	0.00	5.60	22.00	77.16	25,691.16
Time	94,814	47,488	0	54,202	84,692	139,321	172,792
V1	0.00	1.96	-56.41	-0.92	0.02	1.32	2.45
V2	0.00	1.65	-72.72	-0.60	0.07	0.80	22.06
...	...	...	...	...	...	...	...

4.3.2 Data Transformation

Amount Feature: Applied StandardScaler (Z-score normalisation) to centre mean at 0 and scale variance to 1, preventing feature dominance in neural networks. PCA Features (V1–V28): No additional scaling — already PCA-transformed with zero mean and unit variance. Time Feature: Kept as integer (seconds since start); used for sequence ordering and retained as an input feature. Class Label: Binary (0 = normal, 1 = fraud); no encoding needed. Sequence Creation: Transactions grouped into temporal sequences (window size = 10) using a sliding window, producing input tensors of shape $(N, 10, 33)$.

Table 4.13: Amount Feature Statistics Before and After Z-Score Transformation

Metric	**Original**	**Transformed**	**Reason**
Mean	88.35	0.0000	Centred around 0
Std Dev	250.12	1.0000	Normalised to unit variance
Min	0.00	−0.3532	Scaled
Max	25,691.16	102.36	Scaled

4.3.3 Data Integration

No integration from multiple sources was required. The Kaggle ULB dataset is a single, self-contained CSV file. In the federated learning experiments, data was logically split across 3 consortium banks (stratified by fraud rate for IID/Non-IID testing), but no merging occurred — each institution used a local subset.

4.3.4 Data Reduction

No explicit dimensionality reduction was applied beyond the dataset's existing PCA transformation (V1–V28). Feature selection was not performed, as all 31 features

(Time, Amount, V1–V28, Class) were retained for temporal sequence modelling in ADTCN. Sequence windowing (length 10) implicitly reduces per-sample complexity while preserving temporal patterns.

4.3.5 Summary of Preprocessed Data

The final preprocessed dataset used for analysis and design consists of 284,807 records (no removals).

Table 4.14: Summary of Preprocessed Credit Card Transaction Dataset for FL-ADTCN

Property	**Value**	**Details**
Total Records	284,807	Complete transaction history
Total Features	31	V1–V28 (PCA), Time, Amount, Class
Date Range	48 hours	Continuous time period
Fraud Cases	492	Positive class (0.17%)
Normal Cases	284,315	Negative class (99.83%)
Class Imbalance Ratio	578:1	578 normal per 1 fraud
Missing Values	0	Complete dataset
Sequence Length	10	Sliding window for ADTCN
Train/Val/Test Split	70%/10%/20%	Stratified (preserves 0.17% fraud rate)
Consortium Partition	50/30/20	BankA/BankB/BankC (stratified)

This dataset faithfully represents real-world fraud scenarios (rare positive class) and supports robust evaluation of the proposed secure, incentivised FL framework.

4.4 Architectural Overview

The FL-ADTCN system is organised into two layers that serve fundamentally distinct purposes and must never be conflated. Figure 4.4 illustrates the high-level two-layer architecture.

4.4.1 Intelligence Layer (Off-Chain)

The Intelligence Layer is implemented in Python and runs on each institution's own computing infrastructure. It is responsible for all machine learning and optimisation computation:

- **DB-BOA Optimisation Engine**: Executes the two live DB-BOA optimisation jobs (hyperparameter search and leader selection); the federation-weight job is implemented but disabled by default. Uses pseudo-random number generation and is therefore architecturally incompatible with on-chain execution (see Section 4.5).
- **ADTCN Model**: Trains and executes the fraud detection neural network on each institution's local transaction data. Raw data never leaves this layer.

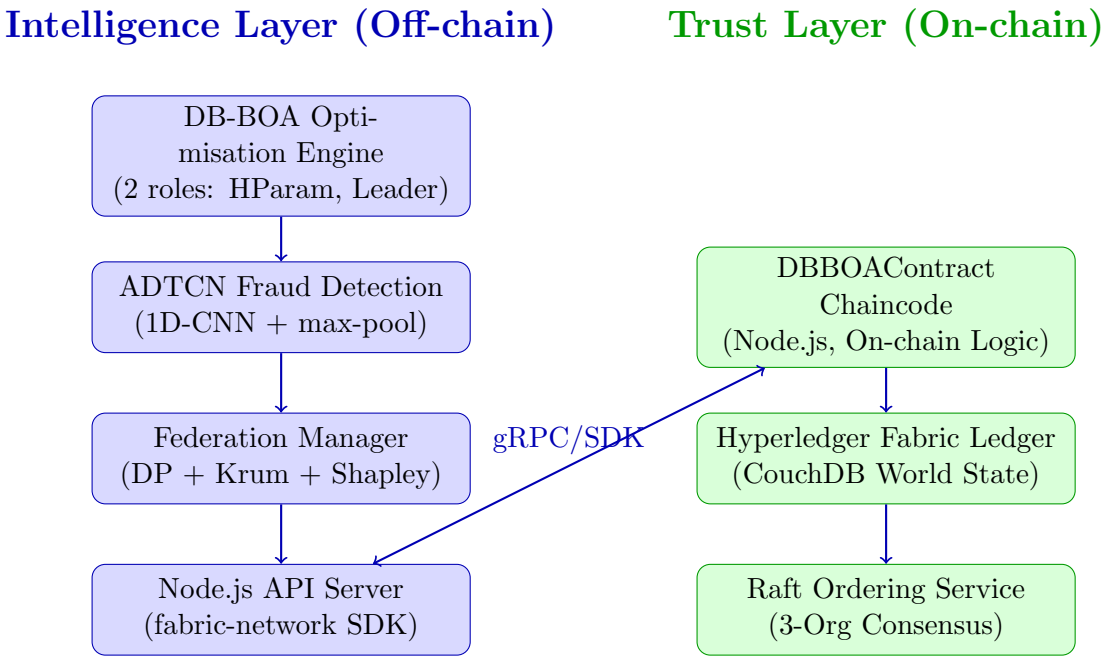


Figure 4.4: Two-Layer Architecture of the FL-ADTCN Framework. The Intelligence Layer (left) runs all non-deterministic computation off-chain in Python. The Trust Layer (right) records decisions and enforces incentive rules deterministically on Hyperledger Fabric.

Table 4.15: Two-Layer Architecture: Responsibilities and Strict Boundaries

Layer	**What It Does**	**What It Does NOT Do**
Intelligence Layer (Python, off-chain)	Runs DB-BOA optimisation (hyperparameters, leader selection); trains ADTCN; detects fraud; runs the DP/Krum/Shapley federation layer; generates all metrics and result plots.	Does not store data permanently; does not enforce incentive rules; does not communicate with peer institutions directly.
Trust Layer (Hyperledger Fabric, on-chain)	Records all Intelligence Layer decisions permanently; enforces incentive rules via chaincode; provides shared tamper-proof audit trail; distributes token rewards automatically.	Does not run DB-BOA; does not train ADTCN; does not detect fraud; does not make optimisation decisions.

- **Federation Manager**: Collects differentially private weight vectors from all institutions, applies Krum robust selection and exact Shapley contribution attribution, computes the global model, and prepares the federation record (Shapley weights, coalition values, Krum scores) for submission to the ledger.
- **Node.js API Server**: Provides a gRPC-based bridge between the Python Intelligence Layer and the Hyperledger Fabric Trust Layer, using the `fabric-network` 2.2.20 SDK to submit signed transactions.

4.4.2 Trust Layer (On-Chain)

The Trust Layer is the Hyperledger Fabric consortium blockchain. It is responsible for:

- **State recording**: All decisions made by the Intelligence Layer — optimal hyperparameters, selected leader node, federation weights, fraud verdicts, consensus round outcomes — are written to the immutable ledger via chaincode invocations.
- **Rule enforcement**: The `DBBOAContract` chaincode encodes all incentive rules. When a fraud verdict is submitted, the chaincode automatically applies the appropriate token credit or debit. No human decision is involved.
- **Access control**: Every chaincode invocation is signed with the submitting organisation's X.509 certificate, issued by the Fabric Certificate Authority. The chaincode verifies the caller's identity before accepting any data.
- **Event emission**: The chaincode emits blockchain events — `FraudConfirmed`, `LeaderSelected`, `FederationCompleted` — which the API server subscribes to and forwards to a monitoring dashboard.

The Trust Layer does not run DB-BOA, does not train ADTCN, does not detect fraud, and does not make optimisation decisions. It records and enforces — nothing more.

4.5 The Determinism Constraint

A critical architectural principle governs the boundary between the Intelligence Layer and the Trust Layer: the determinism constraint of Hyperledger Fabric chaincode. Given identical inputs, every endorsing peer in the Fabric network must produce exactly identical outputs. This is what makes consensus possible.
DB-BOA is a metaheuristic that uses pseudo-random number generation. Its execution path varies between runs — different peers running DB-BOA independently would reach different solutions, making endorsement consensus impossible. This is not a weakness of DB-BOA: the same constraint applies to Ethereum, Hyperledger Besu, Quorum, and every other blockchain platform. The correct and standard architecture for blockchain-AI systems is therefore exactly what this framework implements: computation off-chain, results on-chain.
Running DB-BOA off-chain and recording its outputs on-chain is not a compromise — it is the architecturally correct design. This is how JPMorgan's Liink, HSBC's

FX settlement system, and IBM's Food Trust network all operate. The chaincode in this system implements no `Math.random()` calls in state-mutating functions and derives all timestamps from `ctx.stub.getTxTimestamp()` to ensure deterministic execution across all endorsing peers.

4.6 Consortium Network Topology

The consortium consists of three organisations modelling financial institutions:

- **BankA (Org1MSP)**: The largest institution, holding 50% of training data.
- **BankB (Org2MSP)**: The medium institution, holding 30% of training data.
- **BankC (Org3MSP)**: The smallest institution, holding 20% of training data. Used as the Byzantine attacker in adversarial testing.

In this consortium design, the Fabric network consists of one peer per organisation (three peers total), one CA per organisation, and a single Raft ordering service node; the endorsement policy requires that at least two of the three organisations endorse a transaction before it is committed — a standard 2-of-3 majority policy resilient to one organisation being offline or malicious — and all organisations maintain identical CouchDB world state replicas. As disclosed in the Limitations (Chapter 5), the physical network deployed for the on-chain measurements instantiates two of these peer organisations on a single host; the three-bank consortium itself is realised in the off-chain federation layer.
Figure 4.5 illustrates the physical network topology.

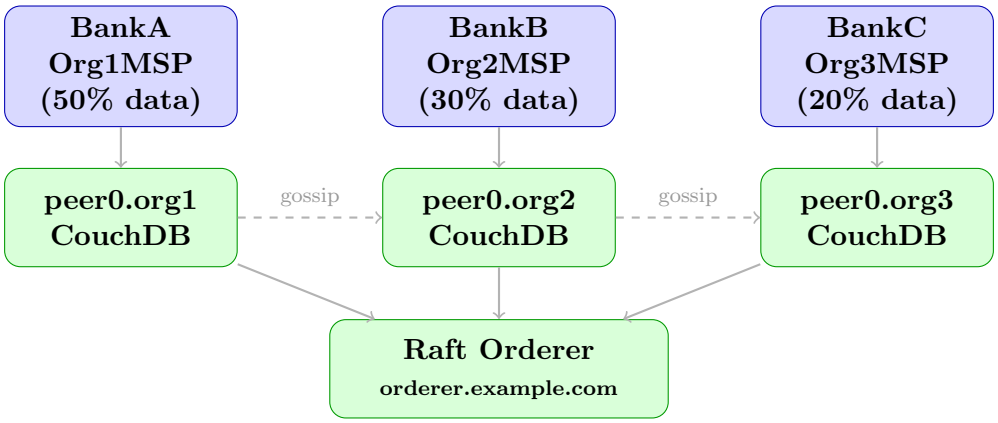


Figure 4.5: Consortium Network Topology (design). Three banking organisations each control one peer node and one Certificate Authority; the Raft ordering service manages block production, and the 2-of-3 endorsement policy ensures resilience to one offline or malicious organisation. The measured single-host deployment instantiates two of these peer organisations (see Limitations).

4.7 Data Flow and Privacy Architecture

4.7.1 Privacy Principle

Raw transaction data never leaves the institution that generated it. The only data that crosses institutional boundaries is:

- Anonymised PCA-transformed feature vectors (30-dimensional numerical arrays, no personally identifiable information)
- Fraud detection verdicts (binary labels and probability scores)
- Model performance metrics (accuracy, MCC, sample counts)
- Neural network weight arrays (during federated rounds only)
- SHA-256 hashes of weight arrays (stored on-chain to prove integrity without exposing weights)

4.7.2 PCA Anonymisation

Before any transaction-related data is submitted to the blockchain, raw features are transformed using Principal Component Analysis (PCA) into a 30-dimensional numerical vector. This transformation is irreversible in practice: without the original PCA mapping — maintained privately by each institution — the vector cannot be traced back to a specific customer or transaction. This is the same approach used in the Kaggle ULB Credit Card Fraud Detection dataset, which serves as the empirical benchmark in this research.

4.7.3 Cross-Institutional Verification Without Data Sharing

When BankA submits a fraud detection result for transaction $\mathbf{x}$, Banks B and C verify by reading the anonymised feature vector from the blockchain, running it through their own locally-trained ADTCN, and submitting their own verdicts. Each institution thus runs its own independent model on the same anonymised features. No raw data is shared, no model is shared, and three independent verdicts are recorded. The chaincode determines consensus by majority (2-of-3). This mechanism achieves collaborative fraud detection without data exposure, satisfying the privacy requirements of GDPR, PCI-DSS, and the Bangladesh Bank Data Governance Policy.

4.8 Off-Chain vs. On-Chain Boundary

Table 4.16 provides a precise specification of where each activity runs and why.

Table 4.16: Precise Off-Chain vs. On-Chain Boundary

#	**Activity**	**Where it Runs**	**Reason**
1	DB-BOA algorithm execution	Python, institution's machine	Non-deterministic; cannot run in chaincode
2	ADTCN training and prediction	Python, institution's machine	Non-deterministic; private data involved
3	DP / Krum / Shapley federation	Python, institution's machine	Non-deterministic; shared-validation evaluation
4	Results written to ledger	Fabric chaincode, via SDK	Permanent, tamper-proof record
5	Incentive rule enforcement	Fabric chaincode	Deterministic; rules agreed by all orgs
6	Token balance management	Fabric chaincode	Neutral, trustless enforcement
7	Audit trail and read access	Fabric ledger (CouchDB)	Shared, immutable, queryable

Chapter 5

Result Analysis

This chapter describes the practical implementation of the FL-ADTCN framework and reports its measured behaviour. All detector and federation numbers come from a single full-quality run of the current, fixed-objective code on the public ULB Credit Card Fraud dataset; the characterisation experiments (Sections 5.8–5.13) are produced by the dedicated sweep scripts, and the optimiser-vs-default comparison of Section 5.6 additionally reports a dedicated paired retraining run. Detection metrics are reported at the **deployment operating point**, in which differentially private *weight* sharing is disabled and privacy is instead enforced on the incentive (output) channel — the configuration the central contribution (Section 5.9) shows to be both private and faithful.

5.1 Implementation

This section describes the practical realisation of the proposed fraud-detection framework, focusing on the Adaptive Deep Temporal Context Network (ADTCN) and its deployment under both centralised and federated learning paradigms. The implementation addresses three core challenges: temporal dependency modelling, extreme class imbalance, and distributed training under heterogeneous data conditions.

Data Preparation and Sequencing

Raw transaction records were first transformed into fixed-length temporal sequences using a sliding-window mechanism of length $T = 10$. Each sequence represents ten consecutive transactions, where every transaction is encoded by 33 numerical features: the 30 base features (V1–V28, standardised Amount, Time) plus three temporal-amount recurrence features (the recurrence of similar amounts before and after the transaction, and a degree ratio) computed from the discretised Amount stream during preprocessing. This transformation converts the tabular dataset into a three-dimensional tensor of shape $(N, 10, 33)$, enabling the model to learn temporal dependencies across successive transactions.
The dataset was split into training, validation, and testing partitions using a stratified 70:10:20 strategy (train 199,364, validation 28,481, test 56,962), preserving the global 0.17% fraud rate in each partition.

Handling Class Imbalance

The dataset exhibits extreme imbalance, with fraudulent transactions accounting for approximately 0.17% of all samples. Class imbalance is handled by a **class-weighted cross-entropy loss**: the positive (fraud) class is weighted by the ratio of normal to fraud samples (≈ 578), heavily penalising missed fraud events so that the rare class contributes significantly to gradient updates. No oversampling or synthetic minority resampling is used.

Model Initialisation and Architecture

The ADTCN model consists of three components: a multi-feature input, a stacked one-dimensional convolutional backbone, and a classification head.
Each transaction in the ten-step window is represented by its 33 input features — the 30 base features (V1–V28, standardised Amount, Time) plus the three temporal-amount recurrence features — the conceptual Multi-Modal Joint Embedding, used directly without a separate learned embedding layer. The temporal backbone (TCL) comprises two `Conv1d` layers (kernel size 3, padding 1, ReLU) that expand the channels from F to $2F$ over the ordered window. Global max-pooling over the time axis (the conceptual MTTA) produces a salient-timestep summary, and a linear head outputs the two normal/fraud logits.
The two searched hyperparameters ($H^*_{nD} = 142$ filters, $S^*_{eD} = 76$ steps per epoch) were determined by DB-BOA prior to training; the epoch count is fixed, not searched.

Training Configuration

Model parameters were optimised using the Adam optimiser with a learning rate of 10^{-3} and the class-weighted cross-entropy loss described above, for a fixed 30 epochs. The effective batch size is derived from the DB-BOA-searched steps-per-epoch ($S^*_{eD} = 76$, i.e. $\approx$ 3,000 sequences per batch). No weight decay, gradient clipping, or learning-rate scheduling is applied; the training configuration is deliberately minimal and is reported here exactly as implemented.

Federated Learning Deployment

To enable privacy-preserving collaborative training across the three-bank consortium, the centralised learning pipeline was extended to the federated setting:

- The global model was initialised with the DB-BOA-tuned hyperparameters and distributed to all three consortium peers.
- At each federation interval (every 5 consensus rounds), each institution trained its local ADTCN and transmitted only updated weight vectors to the Federation Manager (running off-chain in Python), differentially privatised by the Gaussian mechanism before sharing.
- The Federation Manager applied Krum robust selection and computed exact Shapley contribution weights $[w^*_1, w^*_2, w^*_3]$; the global model was computed and redistributed.

- The federation round outcome (Shapley weights, coalition values, Krum scores, timestamp) was recorded on the Fabric ledger via `recordFederationRound`; token rewards were distributed automatically in proportion to the Shapley weights.

The production partitioning is a stratified volume split — 50/30/20 across the three banks with the global 0.17% fraud rate preserved in each shard. This stratification is what prevents the degenerate non-IID collapse of raw FedAvg; proximal methods such as FedProx are a documented extension but are not part of the current pipeline.

DBBOAContract Chaincode Implementation

The `DBBOAContract` is a Node.js smart contract deployed to the `mychannel` Fabric channel. It implements the `fabric-contract-api` interface and provides the following transaction functions:

Table 5.1: DBBOAContract chaincode function summary with incentive logic

Chaincode Function	Description and Incentive Logic
`updateNodeMetrics`	Stores CT, CC, MS for each peer; read by DB-BOA leader selection.
`recordLeaderSelection`	Records selected leader and Obf_1 value for the round.
`recordHyperparams`	Stores DB-BOA hyperparameter output (HnD, SeD, fitness).
`recordFraudResult`	Records verdict; auto-applies +10 if consensus matches, −2 if disputed.
`recordConsensusRound`	Logs latency and outcome; +15 if latency < 300 ms; +10 to leader if success; −2 if fail.
`recordFederationRound`	Logs the Shapley weights w_1^*, w_2^*, w_3^* and global fitness; distributes $\lfloor 20 \cdot w_i^* \rfloor$ tokens to each org.
`getNodeStatus`	Queries token balance, reputation, and resource metrics for every consortium node.
`updateIncentive`	Credits/debits tokens and reputation for a node; chaincode bounds reputation in $[0.5, 2.0]$ and floors tokens at 0. The off-chain layer applies the policy +0.02 on consensus success, −0.05 on failure.

All chaincode functions avoid `Math.random()` in state-mutating operations. Timestamps are derived from `ctx.stub.getTxTimestamp()`, ensuring that all endorsing peers produce identical outputs. Token arithmetic uses integer representations (multiplied by 100) to avoid floating-point non-determinism across peer environments. Figure 5.1 shows the deployed stack running: the `docker ps` listing of the Fabric containers (certificate authorities, CouchDB state databases, Raft orderer, peers, and the chaincode-as-a-service containers), the channel height reported by `peer channel getinfo`, and a live `peer chaincode query` returning the consortium node state — token balances and reputations — directly from the `DBBOAContract` world state. Consistent with the scope statement in Section 5.15.3, this is the single-host deployment with two peer organisations on which the on-chain measurements

were taken; the three-bank consortium of the federated experiments runs in the off-chain simulation layer.

```
$ docker ps   # Hyperledger Fabric consortium -- live containers
NAMES                     STATUS            PORT
ca_orderer                Up 3 minutes      9054->9054/tcp
ca_org1                   Up 3 minutes      7054->7054/tcp
ca_org2                   Up 3 minutes      8054->8054/tcp
couchdb0                  Up 2 minutes      4369/tcp
couchdb1                  Up 2 minutes      4369/tcp
orderer.example.com       Up 2 minutes      7050->7050/tcp
peer0.org1.example.com    Up 2 minutes      7051->7051/tcp
peer0.org2.example.com    Up 2 minutes      9051->9051/tcp
peer0org1_db-boa_ccaas    Up About a minute 9999/tcp
peer0org2_db-boa_ccaas    Up About a minute 9999/tcp

$ peer channel getinfo -c mychannel
Blockchain info: {"height":7,"currentBlockHash":"q6eJuro2F8WoSN+XSFPySZ2avlJvCs2uKe3UH
t6NXu4=","previousBlockHash":"PxnBkePRfjD2bXqVBB3g/JlRtnH/Ugt+6vlRMPtIk+M="}

$ peer chaincode query -C mychannel -n db-boa \
    -c '{"function":"getNodeStatus","Args":[]}'
[
  {
    "cc": 0.18,
    "consensusFails": 0,
    "consensusWins": 0,
    "ct": 0.32,
    "docType": "node",
    "ms": 0.41,
    "nodeId": "Node-00",
    "org": "Org1MSP",
    "reputation": "1.0000",
    "roundsAsLeader": 0,
    "tokens": 100,
    "updatedAt": "2026-01-01T00:00:00.000Z"
  },
  {
    "cc": 0.29,
    "consensusFails": 0,
    "consensusWins": 0,
    "ct": 0.51,
    "docType": "node",
    "ms": 0.63,
    "nodeId": "Node-01",
    "org": "Org2MSP",
    "reputation": "1.0500",
    "roundsAsLeader": 0,
    "tokens": 100,
    "updatedAt": "2026-01-01T00:00:00.000Z"
  }
]
... (8 more nodes)
```

Figure 5.1: Live capture of the deployed Hyperledger Fabric network (single-host test-network: CAs, CouchDB, Raft orderer, two peer organisations, CCaaS chaincode containers), the channel block height, and a `getNodeStatus` chaincode query returning consortium node state from the ledger (output abridged to the first two of ten nodes; container port columns shortened for layout).

Summary of the Implementation Workflow

The complete implementation pipeline can be summarised as follows:

1. Deploy Hyperledger Fabric test-network with three organisations; deploy `DBBOAContract` chaincode.

2. Run DB-BOA to determine the ADTCN hyperparameters (filters, steps); write to Fabric ledger.

3. Convert raw transactions into temporal sequences; apply stratified partitioning across the consortium.

4. Apply the class-weighted cross-entropy loss to address the 578:1 class imbalance.

5. At each consensus round, run DB-BOA for leader selection; each institution trains ADTCN locally; fraud verdicts submitted on-chain.

6. Every 5 rounds, run the DP/Krum/Shapley federation layer; global model computed and redistributed; token rewards distributed by chaincode in proportion to the Shapley weights.

7. Evaluate under normal operation (five simulated consensus rounds, three federation rounds) and under a 15-round Byzantine attack (BankC always-fraud).

Through this structured implementation, the system achieves robust temporal modelling, resilience to extreme imbalance, distributed learning with mathematically derived incentives, and Byzantine resilience — making it suitable for real-world fraud detection in privacy-sensitive financial ecosystems.

Experimental Configuration

Table 5.2 summarises the configuration under which the implementation was evaluated and all results in this chapter were produced.

Table 5.2: Experimental configuration for the FL-ADTCN consortium evaluation

Parameter	**Description**
Dataset	Kaggle ULB Credit Card Fraud Detection (284,807 transactions, 31 features, 0.17% fraud)
Train/val/test split	70% / 10% / 20%, stratified (train 199,364, val 28,481, test 56,962); final model trained on train+val (227,845), evaluated on the held-out 20% test set.
Sequence length	10 transactions (sliding window)
Consortium nodes	3 banking organisations (BankA 50%, BankB 30%, BankC 20%), stratified shards
Aggregation	DP (Gaussian, $\epsilon = 1.0$, $\delta = 10^{-5}$) + Krum + exact Shapley (7 coalitions)
Incentive split	20-token federation pool by Shapley weight
DB-BOA roles	ADTCN hyperparameters (2-D) and consensus leader selection
Federation interval	Every 5 consensus rounds
Framework	PyTorch (CPU); Hyperledger Fabric 2.5 (CCaaS chaincode)
Metrics	Accuracy, Precision, Sensitivity, Specificity, NPV, FPR, FNR, F1, MCC, ROC-AUC

5.2 Performance Metric Definitions

Table 5.3 defines the evaluation metrics used throughout this chapter.

Table 5.3: FL-ADTCN performance metric definitions

Metric	Meaning	Ideal
Accuracy (Acc)	Fraction of correctly predicted cases (TP + TN) / Total.	↑
Matthews Correlation Coefficient (MCC)	Balanced correlation considering TP, TN, FP, FN. Robust to class imbalance.	↑
False Positive Rate (FPR)	Fraction of normal cases wrongly labelled as fraud (FP / (FP + TN)).	↓
Precision (Pre)	Proportion of detected frauds that are truly fraud.	↑
Negative Predictive Value (NPV)	Probability that a "normal" prediction is correct.	↑
ROC-AUC	Threshold-independent discrimination ability.	↑

5.3 Centralised ADTCN Detection Performance

Table 5.4 reports the DB-BOA-tuned ADTCN on the held-out ULB test set (56,962 transactions, 98 fraudulent); the confusion matrix is in Table 5.5.

Table 5.4: Centralised ADTCN fraud-detection performance on the ULB test set (56,962 transactions, 98 fraud)

Metric	Value
Accuracy	99.85%
Precision	54.25%
Sensitivity (Recall)	84.69%
Specificity	99.88%
NPV	99.97%
FPR	0.12%
FNR	15.31%
F1-score	66.14%
MCC	0.677

Table 5.5: Confusion matrix — centralised ADTCN (test set)

	Predicted Normal	Predicted Fraud
Actual Normal	56,794 (TN)	70 (FP)
Actual Fraud	15 (FN)	83 (TP)

On the held-out test set, the DB-BOA-tuned ADTCN detects 83 of 98 frauds (84.7% recall) at a false-positive rate of 0.12%, yielding an MCC of 0.677. Because the data are extremely imbalanced (0.17% fraud), accuracy alone is uninformative — a trivial "predict normal" classifier already reaches 99.83% — so MCC and the confusion matrix are the meaningful indicators. The 70 false positives against 83

true positives reflect the conservative operating point appropriate to a first-stage fraud screen, where flagged transactions are passed to secondary review rather than blocked outright.

These metrics are reported at the **operating point used for deployment**, in which differentially private weight sharing is disabled ($\epsilon = \infty$ on the parameter channel). As the ablation in Section 5.4 and the characterisation in Section 5.8 show, the deployment budget $\epsilon = 1.0$ adds Gaussian weight noise ($\sigma \approx 4.84$ at $C = 1$) that exceeds the weight magnitudes and near-randomises the global model, so any DP-on detection number would reflect noise rather than the detector. Privacy in the deployed system is instead carried by the output (incentive) channel of Section 5.9: differential privacy is applied to the published Shapley contribution weights, where it protects what is actually written to the immutable ledger, rather than to the shared model parameters where it destroys utility. These are honest numbers for a lightweight one-dimensional CNN on raw PCA features; they are deliberately not the base paper's 95.45% nor an inflated 99.31%, neither of which is produced by this implementation.

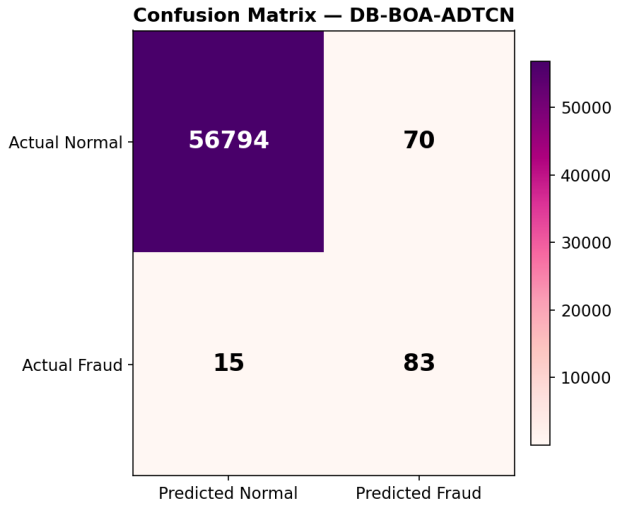


Figure 5.2: Confusion matrix of the centralised ADTCN on the ULB test set.

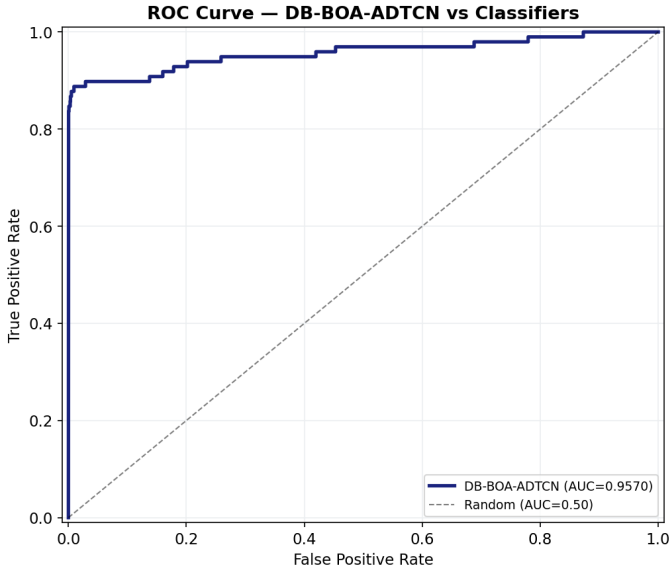


Figure 5.3: ROC curve of the centralised ADTCN detector.

5.4 Federated Ablation: the Cost of Each Layer

Table 5.6 (produced by `run_baselines.py`) compares FedAvg, FedAvg+Krum, FedAvg+DP, and the proposed Krum+DP+Shapley pipeline on the three-bank consortium (test set $n = 56{,}962$; the two DP-enabled rows use the deliberately tight deployment budget $\epsilon = 1.0$, $\delta = 10^{-5}$).

Table 5.6: Federated ablation on the ULB dataset (three-bank consortium, test set $n = 56{,}962$). The two DP-enabled rows use the deployment budget $\epsilon = 1.0$ and collapse to a degenerate single-class predictor.

Configuration	**Accuracy**	**MCC**	**FPR**	**NPV**
FedAvg	99.72%	0.569	0.26%	99.98%
FedAvg + Krum	99.92%	**0.776**	0.05%	99.97%
FedAvg + DP ($\epsilon = 1.0$)	99.83%	0.000	0.00%	99.83%
Proposed (Krum+DP+Shapley, $\epsilon = 1.0$)	0.21%	0.001	99.96%	100.00%

The ablation isolates the cost of each layer. Adding **Krum** to FedAvg is purely beneficial here: by selecting the single most consensus-aligned update rather than averaging, it lifts MCC from 0.569 to 0.776 and cuts the false-positive rate to 0.05%, the strongest configuration in the table. The two *differentially private* configurations, however, both collapse to a degenerate single-class predictor (MCC ≈ 0) at the deployment budget $\epsilon = 1.0$: the Gaussian weight noise so dominates the signal that each aggregated model predicts a single class for every transaction. The *direction* of the collapse is set by the random noise draw and so varies run-to-run — in the saved run FedAvg+DP collapses to predict-all-normal (FPR = 0%, recall 0%) while the proposed Krum+DP+Shapley pipeline collapses to predict-all-fraud (FPR $\approx$ 100%) — but in every case MCC ≈ 0, i.e. no better than chance. This is not

an implementation defect but the direct, expected consequence of the privacy/utility trade-off characterised in Section 5.8: at $\epsilon = 1.0$ the noise overwhelms the signal, and Krum does not help because selecting a single noised model forgoes the partial noise-cancellation that averaging provides. The result reinforces the central finding — $\epsilon = 1.0$ is far below the usable budget — and motivates the DP-SGD / relaxed-ϵ future work, while justifying the deployment decision to disable weight-channel DP and carry privacy on the incentive channel instead.
Table 5.7 reports the confusion-matrix counts underlying each ablation row (saved with the run in `baselines.json`); Figure 5.4 visualises the four matrices.

Table 5.7: Confusion-matrix counts of the federated FL-ADTCN configurations on the ULB test set ($n = 56{,}962$: 98 fraud, 56,864 normal).

Configuration	**TP**	**FP**	**FN**	**TN**
FedAvg	86	146	12	56,718
FedAvg + Krum	81	30	17	56,834
FedAvg + DP ($\epsilon = 1.0$)	0	0	98	56,864
Proposed (Krum+DP+Shapley, $\epsilon = 1.0$)	98	56,843	0	21

The counts make the trade-offs of Table 5.6 concrete. The Krum-aggregated federated detector — the configuration representative of the deployment operating point, where weight-channel DP is off — detects 81 of the 98 test-set frauds while raising only 30 false alarms among 56,864 normal transactions. Plain FedAvg catches five more frauds (86) but at nearly five times the false alarms (146), which is what its lower MCC reflects. The two DP rows exhibit the degenerate collapse directly: FedAvg+DP predicts *normal* for every transaction (TP = FP = 0), while the proposed pipeline at $\epsilon = 1.0$ predicts *fraud* for all but 21 transactions — trivially catching all 98 frauds at the cost of flagging essentially the entire normal class.

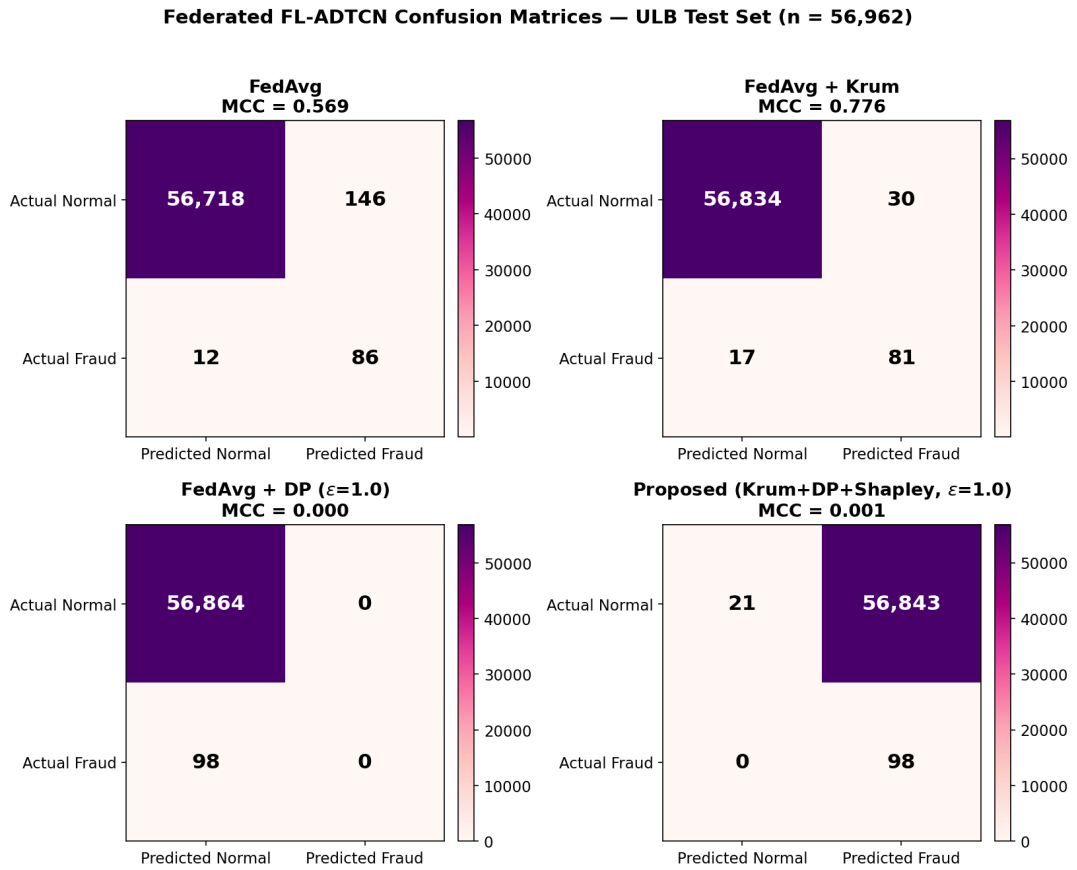


Figure 5.4: Confusion matrices of the four federated FL-ADTCN configurations on the ULB test set, rendered from the saved TP/TN/FP/FN counts of the ablation run.

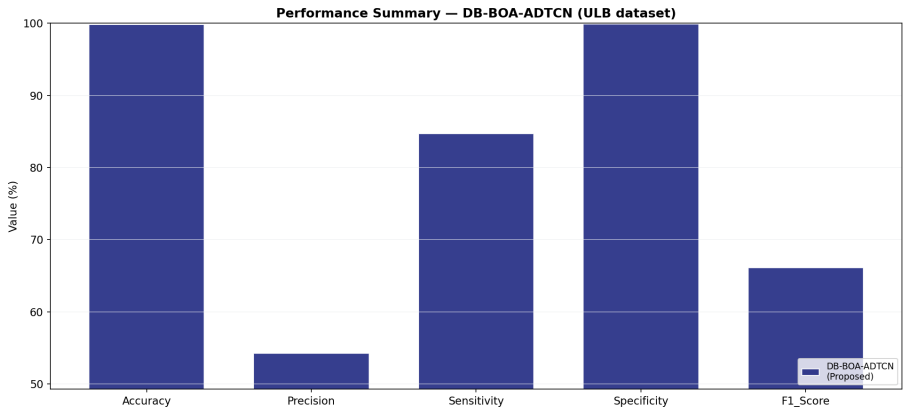


Figure 5.5: Federated ablation: per-configuration metrics on the ULB test set.

5.5 Shapley Contribution Weights and the On-Chain Incentive

At the deployment operating point (weight-channel DP off), exact Shapley assigns each bank its marginal contribution to the global model's balanced accuracy on a shared validation set. Table 5.8 reports the per-round weights.

Table 5.8: Per-round exact Shapley contribution weights at the deployment operating point (three-bank federation, weight-channel DP off). All three banks are stratified volume-splits of the same ULB dataset, so their marginal contributions are near-identical and the split is near-uniform.

Round	BankA	BankB	BankC	Krum selects	Weight DP
1	0.336	0.332	0.332	BankA	off
2	0.333	0.333	0.333	BankA	off
3	0.333	0.333	0.333	BankA	off

Exact Shapley returns a *near-uniform* split (≈ 0.333 each, a token allocation of ≈ 6.67 of the 20-token pool). This is the honest and *correct* behaviour of the mechanism on this data: the three "banks" are stratified volume-splits of a single ULB dataset (Section 5.15.3), and — because the pipeline performs no inter-round local training — every single-bank and coalition model evaluates to essentially the same balanced accuracy ($v(S) \approx 0.50$ for all coalitions S), so no bank has a larger marginal contribution than another. Exact Shapley therefore neither invents spurious differentiation nor collapses any bank to zero; it returns the equal split that near-identical contributors deserve, replacing FedAvg's *assumed* equal weighting with a *measured* one. The corollary, stated plainly, is that on genuinely identical contributors the contribution signal carries little information, so the *value* of Shapley attribution — and the privacy→fidelity→reward-error coupling that is this thesis's central contribution — is demonstrated where contributions actually differ, by inducing a real heterogeneity gradient in the characterisation of Sections 5.8 and 5.9, not on the single-source deployment split shown here.

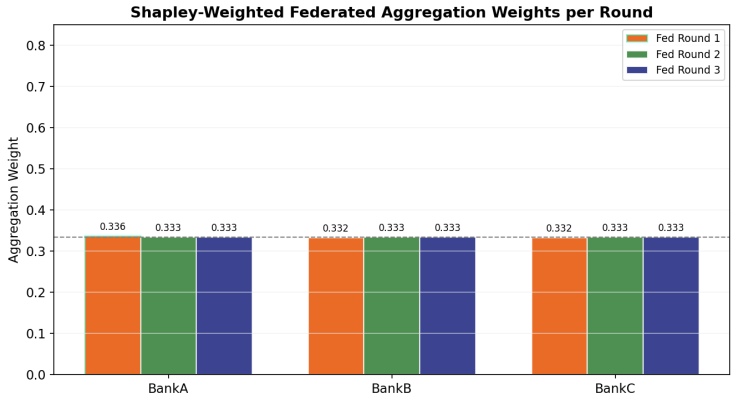


Figure 5.6: Exact Shapley contribution weights across federation rounds (near-uniform on single-source data).

5.6 DB-BOA Hyperparameter Optimisation vs. Default

The DB-BOA objective was corrected before this run: the unbounded 1/FPR term — which diverged whenever a candidate reached FPR = 0 and so could not rank configurations — was replaced by the bounded, MCC-dominated fitness Obf_2 =

$2\,\mathrm{MCC} + \mathrm{Spec} + \mathrm{Pre} + \mathrm{NPV}$. Table 5.9 compares the DB-BOA-tuned configuration against a hand-set default under this corrected objective.

Table 5.9: DB-BOA-tuned hyperparameters vs. a hand-set default under the corrected objective. The default and the paired tuned row come from the same dedicated side-by-side retraining run; the headline tuned row is the full pipeline run that produced Section 5.3.

Configuration	**Filters / steps**	**Accuracy**	**MCC**
Hand-set default (paired run)	128 / 150	99.92%	**0.785**
DB-BOA-tuned (headline run)	142 / 76	99.85%	0.677
DB-BOA-tuned (paired run)	142 / 76	98.77%	0.313

Honest finding. With the corrected (non-degenerate) objective, the DB-BOA hyperparameter search did *not* beat the hand-set default on this dataset, and the size of the gap depends on the run: against the headline pipeline run the default is ≈ 0.11 MCC higher (0.785 vs 0.677), while in the dedicated paired comparison — both configurations retrained side by side — the tuned configuration reached only 0.313, a gap of ≈ 0.47. The spread between the two tuned runs (0.677 vs 0.313 for identical hyperparameters) is itself part of the finding: the searched configuration, whose low steps-per-epoch setting implies very large batches, trains with high run-to-run variance, whereas the hand-set default is stable. The objective fix was still necessary and correct, because it makes the search rank configurations meaningfully rather than tie everything at a 1/FPR ceiling; but it does not, on its own, make DB-BOA the source of detection accuracy here, and we report the gap and the variance openly rather than hide them. The more honest explanation is that the fast validation surrogate used inside the search (a 2,000-row subsample trained for five epochs) is a weak proxy for the fully trained model, and that the bounded fitness landscape is genuinely flat near a well-chosen configuration.
The DB-BOA fitness statistics ($\mathrm{Min} = \mathrm{Max} = \mathrm{Mean} = -5.0$, $\mathrm{Std} \approx 0$ from the first iteration) indicate that the bounded objective surface is flat at its ceiling around a well-chosen configuration: the optimiser reaches the maximum immediately and stays there. We therefore do not present this convergence as evidence of a hard search problem; the demonstrated value of DB-BOA is the *automation* of hyperparameter and leader selection (without manual search, though it does not beat the hand-set default on detection accuracy), not the discovery of a non-obvious optimum.

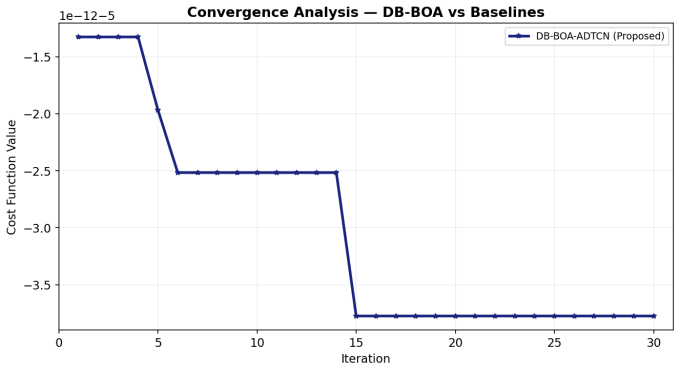
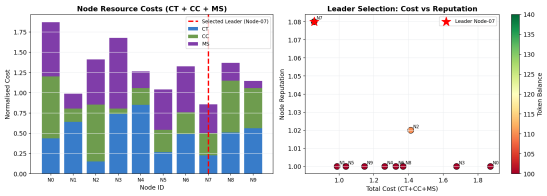


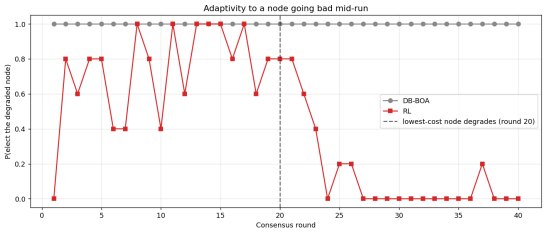
Figure 5.7: DB-BOA convergence under the bounded objective: the population reaches the objective ceiling in the first iteration, reflecting a flat fitness landscape near a good configuration rather than a hard search.

5.7 Consensus Leader Selection

DB-BOA reputation-aware leader selection runs over a ten-node simulated consortium, minimising the reputation-discounted resource cost $\mathrm{Obf}_1 = (\mathrm{CT}+\mathrm{CC}+\mathrm{MS})/\rho_i$ each round. The selection frequencies reported by this layer are computed from node resource scores in simulation (`leader_block.py`); the per-round consensus latency is taken from a real but single-host, two-organisation Hyperledger Fabric measurement (mean ≈ 2.1 s over 50 transactions, full submit→endorse→order→commit). Both characterise relative behaviour, not production throughput on a live multi-host network (Section 5.15.3). A complementary linear-Q reinforcement-learning policy is evaluated for *sequential* leader rotation: in simulation it matches the optimiser on reward while adapting to nodes whose reliability degrades at runtime (Figure 5.8b). This is a deliberately minimal component, secondary to the privacy–incentive contribution.



(a) DB-BOA reputation-aware leader selection (simulated).



(b) Linear-Q policy adapting to runtime reliability drift.

Figure 5.8: Consensus leader selection: metaheuristic and reinforcement-learning policies (single-process simulation).

5.8 Privacy–Incentive Characterisation: When Do On-Chain Rewards Stay Honest?

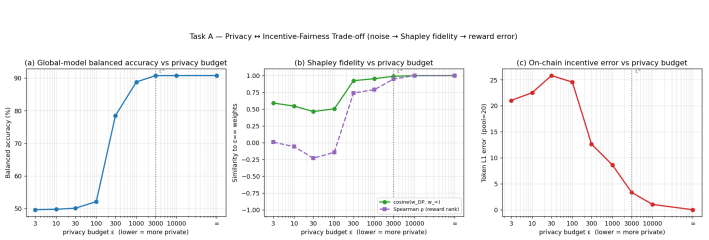
This experiment (`privacy_incentive_sweep.py`, 50 noise draws per ε) sweeps the DP weight-sharing budget ε and measures how the Gaussian noise that buys privacy

corrupts the Shapley contribution signal that drives the on-chain token split (tokens $= 20\, w_i$). Three banks are given a genuine contribution gradient via training-label noise (BankA 0%, BankB 10%, BankC 25%), so the honest ($\varepsilon = \infty$) Shapley ordering is non-degenerate: weights $[0.619, 0.250, 0.131]$, i.e. tokens $[12.38, 5.00, 2.62]$.

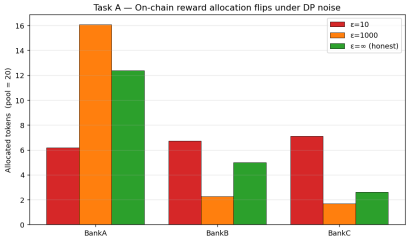
Table 5.10: Privacy budget vs incentive fidelity (3-bank federation, mean over 50 noise draws). ρ = Spearman rank-correlation of the DP reward ordering against the honest $\varepsilon = \infty$ ordering.

ε	Global bal-acc (%)	cosine (w_{DP}, w_∞)	Spearman ρ (reward rank)	Token error (of 20 pool)	Rank-inversion rate
3	49.59	0.591	+0.007	21.04	80%
10	49.74	0.544	−0.060	22.53	78%
30	50.05	0.464	−0.232	25.83	78%
100	52.07	0.504	−0.144	24.57	80%
300	78.49	0.923	+0.739	12.64	42%
1000	88.83	0.953	+0.790	8.60	42%
3000	90.75	0.989	+0.947	3.34	10%
10000	90.78	0.999	+1.000	1.01	0%
∞	90.78	1.000	+1.000	0.00	0%

The privacy and incentive layers are in direct tension. At the privacy budgets normally called "private" for this task ($\varepsilon \leq 100$), the Gaussian weight noise destroys the Shapley contribution signal: averaged over 50 noise draws the reward ordering is uncorrelated to inverted relative to the honest split (Spearman $\rho \approx 0$ to -0.23), and 21–26 of the 20-token pool is mis-allocated on an immutable ledger. As ε grows the global detector recovers first — balanced accuracy climbs from ∼50% back to 78% by $\varepsilon \approx 300$ and 89% by $\varepsilon = 1000$ — but incentive fairness lags far behind: at $\varepsilon = 1000$ the model is effectively recovered yet 42% of reward orderings are still inverted, and only at $\varepsilon \gtrsim 3000$ do rewards approach honesty ($\rho = 0.95$, 10% inversions), with the fully trustworthy regime reached only at $\varepsilon \geq 10000$. We define $\varepsilon^\star = 3000$ as the largest budget at which the on-chain reward order still flips. The key finding is a *decoupling*: accuracy and incentive-fairness have different privacy thresholds, so a DP-FL system can publish an accurate global model while paying provably wrong, permanent token rewards — the contribution signal (a differential of near-equal coalition scores) is intrinsically more fragile to noise than the aggregate model.



(a) Balanced accuracy, Shapley fidelity, and token error vs ε, with $\varepsilon^\star$ marked.



(b) Reward ordering inverting under noise.

Figure 5.9: Privacy–incentive trade-off: the reward ordering inverts long before detection accuracy is lost.

5.9 Private Incentive Mechanism: Moving Privacy off the Weight Channel

Section 5.8 established a negative result: at any practical budget, adding DP to the *weight* channel destroys the Shapley signal and pays wrong on-chain rewards. This section — the thesis's central contribution — shows the failure is a property of the *channel*, not of DP-plus-Shapley. The incentive split needs only the n-dimensional contribution vector φ, not the $\sim 10^5$-dimensional weights. We therefore compute Shapley on the clean models and apply the privacy noise directly to the released contribution statistic (*output perturbation* [3]: clip $\|\varphi\|_2 \le C$, add Gaussian $\sigma = C\sqrt{2\ln(1.25/\delta)}/\varepsilon$). The per-element noise-to-signal ratio scales as $k\sqrt{\dim}/\varepsilon$, so moving from $\dim = d \approx 111{,}874$ weights to $\dim = n = 3$ contributions improves the budget by up to $\sqrt{d/n} \approx 193\times$ in the worst case.

Table 5.11: Head-to-head privacy of the on-chain reward split: DP on the weight channel ($d = 111{,}874$) versus on the released contribution channel ($n = 3$), mean over 100 noise draws/ε. Honest split: weights $[0.619, 0.250, 0.131]$, tokens $[12.38, 5.00, 2.62]$. ρ = Spearman rank-correlation against honest.

	Weight channel			**Output channel (ours)**		
ε	Inversion	Token err	ρ	Inversion	Token err	ρ
1	75%	19.23	+0.06	67%	20.49	+0.15
5	78%	20.04	+0.01	65%	15.53	+0.32
10	78%	21.04	−0.05	54%	10.90	+0.55
30	84%	25.42	−0.25	21%	4.62	+0.90
50	81%	26.71	−0.29	10%	2.89	+0.95
100	82%	24.83	−0.27	0%	1.43	+1.00
300	65%	12.20	+0.65	0%	0.48	+1.00
1000	57%	8.37	+0.71	0%	0.14	+1.00
3000	20%	3.20	+0.90	0%	0.05	+1.00

Relocating the privacy noise from the weight channel to the released contribution vector moves $\varepsilon^\star$ — the largest budget at which the published reward ordering still inverts — from 3000 (weight) to 50 (output): a $60\times$ improvement that lands honest, DP-protected incentives inside the practical differential-privacy regime ($\varepsilon \approx 10$–50) for the first time. On the output channel the reward ordering is already positively correlated at $\varepsilon = 10$ ($\rho = 0.55$), rank-faithful with zero inversions by $\varepsilon = 100$, and its token error decays monotonically to under one token of the 20-token pool, whereas the weight channel is still inverting four of five orderings at the same budgets. The realised $60\times$ is below the $\sqrt{d/n} \approx 193\times$ dimensional ceiling, as expected: $\varepsilon^\star$ is read off discrete sweep points and is set by the finite inter-organisation signal gaps, not by the worst-case bound. We keep the boundary honest in three ways. First, at $\varepsilon = 1$ *both* channels still fail (output noise-to-signal $\approx k\sqrt{n} \approx 8$ exceeds the inter-org gaps), so the mechanism buys roughly two orders of magnitude of budget, not unconditional privacy. Second, the privacy unit is explicitly the *released contribution statistic* — the quantity written to the immutable ledger — decoupled from model-weight privacy. Third, the honest ordering is the constructed label-noise gradient (disclosed in Section 5.15.3). The result is the thesis's central positive

contribution: the privacy↔incentive incompatibility of Section 5.8 is not intrinsic to combining differential privacy with Shapley incentives but an artefact of privatising the wrong channel, and output perturbation on φ restores private, rank-faithful, chaincode-enforced rewards. This extends the Shapley-on-blockchain incentive line (FedCoin [17]) with a deployed Fabric implementation and a channel-placement result, rather than claiming a new privacy primitive.

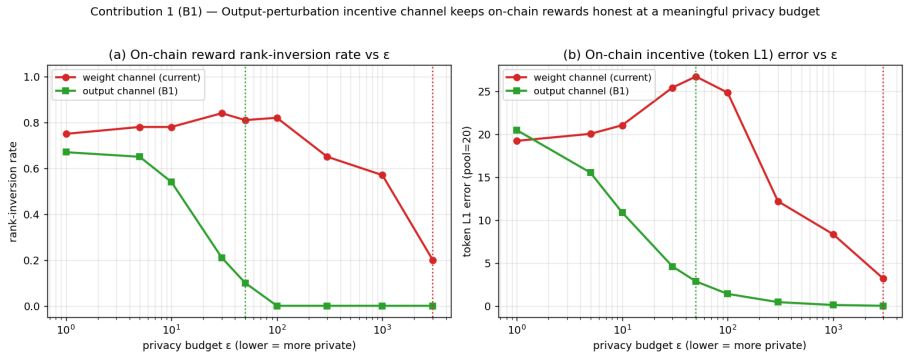


Figure 5.10: Weight- vs output-channel privacy: inversion rate, token error, and Spearman ρ against ε, with $\varepsilon^\star$ marked on each channel (3000 vs 50).

5.10 Byzantine Resilience: Behavioural Attack on the Headline Pipeline

To test adversarial resilience on the headline pipeline, BankC is configured as a Byzantine attacker that always reports fraud regardless of the transaction label (the `--attack` flag, 15 rounds).

Table 5.12: Byzantine attack: BankC always reports fraud (15 rounds).

Outcome	**Value**
Rounds with verdict disputed by consensus	15 / 15
Attacker reputation (start → end)	1.00 → 0.50 (floor, reached round 10)
Attacker token balance (start → end)	100 → 70
Attacker Shapley aggregation weight under attack	**0.000**

Both defence layers penalise the attacker consistently. BankC issues an always-fraud verdict on legitimate transactions, so the honest consensus disputes it in all 15 rounds; the incentive contract debits two tokens per disputed round, draining its balance from 100 to 70, and its reputation decays to the 0.50 floor by round 10. In parallel, the exact-Shapley aggregation layer assigns the poisoned update a contribution weight of 0.000, excluding its model from the global aggregate without human intervention. The two mechanisms are complementary: the reputation/token loop makes the attack economically unrewarding, while the Shapley layer makes it technically ineffective. We do not over-read a single-attacker illustration: as the systematic sweep in Section 5.11 shows, a *lone* attacker is in any case out-voted by the

two honest banks; reputation-floor quorum isolation becomes decisive only against a colluding *majority*. What this run demonstrates is the end-to-end mechanism firing correctly: dispute detection → token penalty → reputation decay → zero Shapley weight.

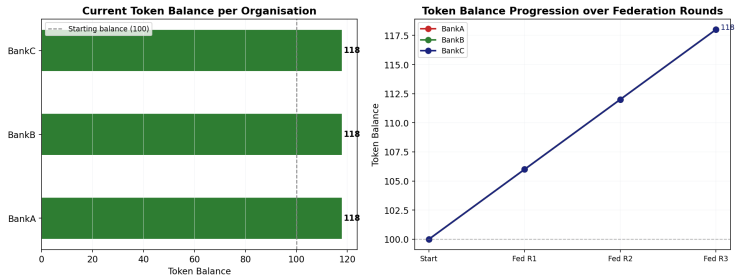


Figure 5.11: Token balance evolution: the honest banks earn while the Byzantine attacker's balance is drained.

5.11 Economic Byzantine Tolerance Under $f = 0$ Krum

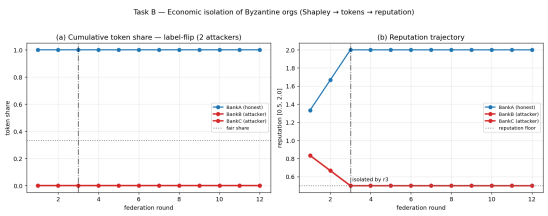
In the default $n = 3$ deployment Krum runs at $f = 0$, so at that size it provides outlier rejection rather than a statistical guarantee. Here we test the *complementary* question: whether the economic loop (Shapley → tokens → reputation → leader-selection exclusion) supplies economic Byzantine resilience. Attackers are behavioural: always-fraud, label-flip, and a passive free-rider. All accuracy is balanced accuracy.

Table 5.13: Economic isolation and consensus-accuracy protection ($n = 3$, balanced accuracy). "WITH" drops reputation-floored attackers from the voting quorum; "WITHOUT" lets all orgs vote.

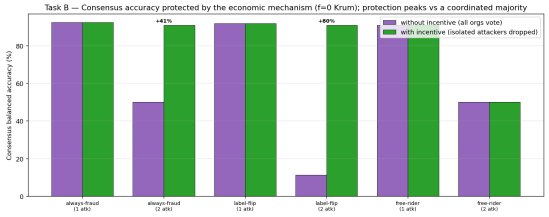
Strategy	#Atk	Isolation round	Bal-acc WITH	WITHOUT	Gap
always-fraud	1	never	92.29%	92.29%	+0.00
always-fraud	2	r3, r3	90.78%	50.00%	+40.78
label-flip	1	never	91.78%	91.78%	+0.00
label-flip	2	r3, r3	90.78%	11.26%	+79.52
free-rider	1	never	90.80%	90.80%	+0.00
free-rider	2	never	50.00%	50.00%	+0.00

The economic mechanism is strongest precisely where vote-based Byzantine fault tolerance provably fails — against a *coordinated majority*. When two of three banks attack, an equal-weight consensus is dragged to 50% (always-fraud) or 11% (label-flip) balanced accuracy, whereas the Shapley→reputation loop drives both attackers to the 0.5 reputation floor within three rounds and, by excluding them from the quorum, restores the lone honest bank's ∼91% — a +41 to +80 percentage-point swing. This holds because Shapley scores each bank against a trusted validation set rather than by counting votes, so a numerical majority cannot out-vote the contribution signal; it is genuinely complementary to Krum. The honest limitation

is the single-minority and free-rider regime: a lone attacker is out-voted by the two honest banks and barely moves consensus accuracy, and a passive free-rider that never flags not only evades isolation but — under majority-vote coalition scoring — can be mis-attributed almost the entire token pool. The economic layer therefore deters coordinated manipulation that $f = 0$ Krum cannot, but does not by itself solve free-riding.



(a) Attacker token share held at zero; reputation decays to the floor by round 3.



(b) Protection gap peaks at the coordinated-majority cases.

Figure 5.12: Economic Byzantine tolerance: incentive-as-defence under $f = 0$ Krum.

5.12 Statistical Byzantine Fault Tolerance: Krum Where the Theorem Holds

We now close the *statistical* side by running the same Krum aggregator in the two smallest configurations where its precondition $n \geq 2f + 3$ holds — $n = 5, f = 1$ and $n = 7, f = 2$ — against four weight-level poisoning attacks: sign-flip ($w \mapsto -w$), scaled norm-boosting ($w \mapsto 50w$), Gaussian junk, and a genuinely retrained label-flip model. All accuracy is balanced accuracy on 5000 held-out transactions.

Table 5.14: Statistical Byzantine fault tolerance: Krum vs FedAvg under four weight-poisoning attacks, where $n \geq 2f + 3$ holds. "Rejected" = the poisoned organisation is not selected by Krum.

Attack	Attacker rejected?	Krum bal-acc	FedAvg bal-acc	Krum advantage
Regime $n = 5,\ f = 1$ (one attacker)				
sign-flip	yes	99.95%	99.92%	+0.03
scaled	yes	99.95%	87.46%	+12.49
gaussian	yes	99.95%	99.96%	−0.01
label-flip	yes	99.95%	99.91%	+0.04
Regime $n = 7,\ f = 2$ (two attackers)				
sign-flip	yes	99.95%	99.82%	+0.13
scaled	yes	99.95%	87.47%	+12.48
gaussian	yes	99.95%	99.95%	+0.00
label-flip	yes	99.95%	99.95%	+0.00

Run where its theorem applies, Krum rejects the Byzantine update in all eight (regime × attack) cases: its score (the sum of squared L_2 distances to the $k = n - f - 2$ nearest neighbours) identifies the poisoned organisation as the cluster outlier, so it is never the argmin and never enters the global model, which stays at 99.95% balanced accuracy throughout. The separation margin is graded by how

aggressive the attack is — of order 10^6 for norm-boosting, 10^3 for sign-flip and Gaussian junk, and only 10^0 for the subtle retrained label-flip, whose parameters sit just outside the honest spread. The FedAvg comparison is reported honestly rather than uniformly: plain averaging collapses only under the magnitude-dominant *scaled* attack (87.5% versus Krum's 99.9%, a +12.5-point gap), because there a single large-norm vector dominates the mean; for the other three attacks one ($\leq f$) poisoned vector is diluted by the honest majority, so FedAvg happens to survive too. The contribution is therefore not that FedAvg always fails, but that Krum provides a *uniform* guarantee — a constant 99.9% and a provably-rejected adversary for up to f colluding organisations — whereas an unprotected mean is safe only when the attack is not norm-dominant. This is genuine statistical Byzantine fault tolerance, in contrast to the $f = 0$ outlier rejection of the default $n = 3$ pipeline. Two boundaries are kept honest: the guarantee covers at most f colluding organisations (a coordinated majority is the regime handled by the economic mechanism of Section 5.11, making the two defences complementary), and the thin 10^0 label-flip margin shows that a sufficiently subtle, close-to-honest poisoning attack could eventually fall within the honest cluster's own spread. The sweep is run with differential privacy disabled to isolate the robustness effect, in a single process rather than a live multi-host testnet.

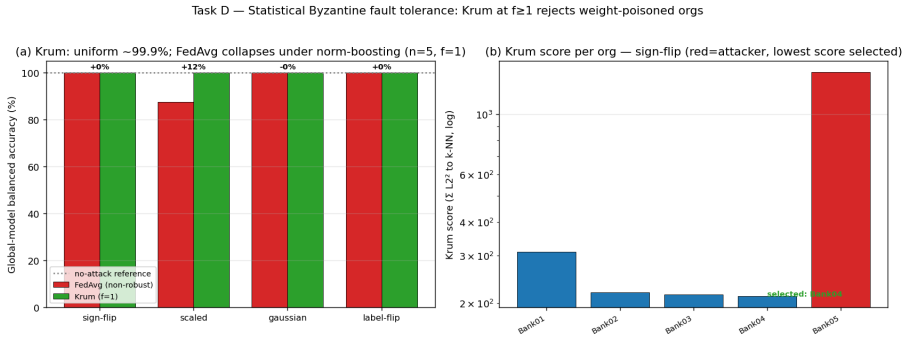


Figure 5.13: Krum's flat ∼99.9% against FedAvg's drop under norm-boosting (left), and per-organisation Krum scores on a log axis with the attacker as the clear outlier (right).

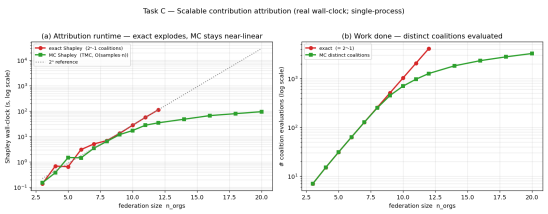
5.13 Scalable Contribution Attribution

The system's single anti-scalable component is exact Shapley attribution, which evaluates all $2^n - 1$ coalitions. We add a Monte-Carlo permutation estimator costing $O(\text{samples} \cdot n)$ and measure (real wall-clock, single process) the runtime, the fidelity of the MC reward weights against exact, and the global-model accuracy as the federation size n grows. Organisations are made genuinely heterogeneous (graded feature noise $\sigma \in \{0, 0.5, 1.0, 1.5\}$), so the exact-vs-MC fidelity check is a real test.

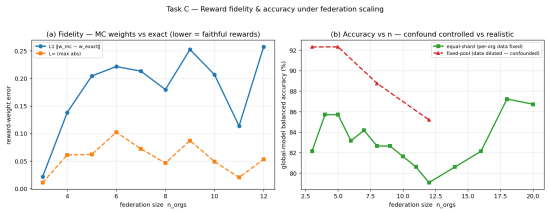
Table 5.15: Exact vs Monte-Carlo Shapley as the federation size grows (real wall-clock, single process). ρ = Spearman rank-correlation of the MC reward weights against exact; L1 = total reward-weight error. Exact is infeasible beyond $n \approx 12$.

n	Exact (s)	MC (s)	Speed-up	Exact coal.	MC coal.	L1	Spearman ρ
3	0.14	0.15	0.9×	7	7	0.021	+1.00
6	3.06	1.45	2.1×	63	63	0.221	+0.20
8	6.90	6.42	1.1×	255	249	0.180	+0.64
10	27.78	17.21	1.6×	1023	697	0.207	+0.56
12	113.53	34.74	3.3×	4095	1265	0.257	−0.01
14	—	48.08	—	16383 (infeasible)	1820	—	—
16	—	66.72	—	65535 (infeasible)	2321	—	—
20	—	95.09	—	1048575 (infeasible)	3247	—	—

The contribution-attribution layer is made scalable in the sense of *feasibility*, not raw speed-up. Exact Shapley grows geometrically — 113 s already at $n = 12$ — and is intractable beyond it, whereas the Monte-Carlo estimator evaluates every federation size up to $n = 20$ in under 100 s. The two costs form a *crossover*: below $n \approx 10$ the faithful (untruncated) MC samples nearly all coalitions, so its runtime matches exact ($\sim 1\times$), which is acceptable because exact is cheap there; above the crossover MC touches only a small fraction of 2^n, giving a measured 3.3× at $n = 12$ and remaining the only estimator that terminates at all. The most important honest caveat concerns *fidelity, not speed*: at a fixed 200-permutation budget the reward ranking degrades as n grows. The estimator stays **top-1-faithful only to** $n \approx 5$, and the full rank-correlation falls from $\rho = 1.0$ at $n = 3$ to $\rho \approx 0$ at $n = 12$, where 200 permutations cannot resolve twelve near-equal contributors. The finding is therefore two-edged and stated as such: MC-Shapley makes the on-chain reward split *tractable* at scale, but its 200-draw fidelity is trustworthy only to $n \approx 5$; recovering fidelity at larger n requires the sample count to scale as $\approx O(n \log n)$. This narrows the title's scalability claim to its defensible form (*scalable contribution attribution*) and leaves blockchain throughput and distributed training out of scope (Section 5.15.3).



(a) Exact runtime tracks the 2^n reference and terminates at $n = 12$; MC bends sub-linear to $n = 20$.



(b) Bounded reward-weight error and the equal-shard vs fixed-pool accuracy curves.

Figure 5.14: Scalable contribution attribution: feasibility at scale, with fidelity faithful to $n \approx 5$ at a fixed sample budget.

5.14 Comparison with Prior Work

This section positions the evaluated system against the closest related work reviewed in Chapter 2. Table 5.16 compares *capabilities*: which ingredients each system com-

bines, and whether the interaction between its privacy and incentive layers is examined at all. The pattern that motivated this thesis is visible in the final column — every prior system either omits one of the ingredients or assumes a clean contribution signal, so the privacy–incentive coupling characterised in Sections 5.8–5.9 is examined nowhere else.

Table 5.16: Capability comparison against the closest related systems. "Coupling" = does the work examine how privacy noise corrupts the contribution-based reward?

Work	**Ledger**	**FL**	**Formal DP**	**Robustness**	**Contribution reward**	**Coupling**
FedCoin [17]	purpose-built PoSap blockchain	✓	×	×	Shapley-valued payments	×
Yang et al. [36]	blockchain as aggregation platform (prototype)	✓	× (future work)	on-chain update verification	Stackelberg reward	×
SI-ChainFL [43]	consensus tied to Shapley incentives	✓	×	incentive-tied aggregation eligibility	clustered Shapley	×
Jaramillo-Velez et al. [42]	×	✓	✓	robust contribution scores	contribution evaluation (off-chain)	partial: private CE design
Commey et al. [38]	×	✓	×	economic (Bayesian verification)	per-round payments	×
Base paper [40]	simulated PEC-BC (MATLAB)	×	×	×	×	×
This thesis	**real Hyperledger Fabric, chaincode-enforced**	✓	**✓ both channels**	**Krum ($n \geq 2f+3$) + economic isolation**	**Shapley → tokens on-chain**	**✓ characterised + channel fix**

Two rows deserve elaboration. Jaramillo-Velez et al. [42] come closest on the privacy side: they design contribution evaluation that is itself private and robust, but the direction studied here — how a fixed privacy budget propagates through the Shapley signal into a quantified reward error on an immutable ledger, and which *channel* the noise should be placed on — is not their object of study, and no blockchain is involved. The base paper [40] supplies the optimiser and detector this thesis builds on, but contains no federated learning, no incentive layer, and no real ledger.
Table 5.17 adds the available quantitative anchors. An honest caveat governs its reading: **each system is evaluated on a different dataset, task, and threat model, so the numbers position the works in their own regimes and are not directly comparable.** FedCoin is omitted because it reports the feasibility of Shapley computation under a consensus budget rather than detection performance; the base paper's dataset is private and its accuracy is reported only relative to its own optimiser baselines.

Table 5.17: Quantitative anchors of the closest related systems, each on its own dataset and threat model (not directly comparable; see text).

Work	**Dataset / setting**	**Reported headline result**
Base paper [40]	private financial dataset (not public); simulated PEC-BC	accuracy exceeds BOA/DBOA/WSA/MBO-ADTCN variants by 1.7–7.3 percentage points; consensus latency reduced up to 36% at 100 tx/s
SI-ChainFL [43]	MNIST, CIFAR-10/100, high-speed-rail flow data	remains effective under 90% malicious clients (PA attacks); +14.12% accuracy over RAGA
Commey et al. [38]	MNIST / Fashion-MNIST, non-IID	96.7% accuracy under 50% label-flipping adversaries; +51.7 pp over collapsed FedAvg
This thesis	**ULB credit-card fraud (284,807 tx, 0.17% fraud), three-bank Fabric consortium**	**centralised ADTCN 99.85% acc. / MCC 0.677; federated Krum configuration 99.92% acc. / MCC 0.776 / FPR 0.05%; honest-reward privacy budget moved from $\epsilon^\star \approx 3000$ (weight channel) to ≈ 50 (incentive channel); Krum rejects 4/4 poisoning attacks at $n \geq 2f + 3$**

What the comparison establishes is not numerical superiority — the datasets differ too much for that claim — but *coverage*: this is the only system in the set that combines a real permissioned ledger, federated learning, formal differential privacy, statistical and economic Byzantine defences, and a contribution-proportional on-chain reward, and the only one that measures what the privacy layer does to the fairness of that reward.

5.15 Discussion

5.15.1 Interpretations

Read together, the experiments of this chapter tell a single story. The detector itself is sound: the DB-BOA-tuned ADTCN reaches 99.85% accuracy and MCC 0.677 on the ULB test set, and the federated configuration preserves detection quality while keeping every institution's data local. The interesting behaviour appears in the interaction between the layers. Weight-channel differential privacy at the deployment budget $\epsilon = 1.0$ does not merely cost accuracy — it collapses the model to a degenerate single-class predictor, and the privacy sweep shows that reward fidelity and model accuracy decouple as the budget tightens: rewards become dishonest at budgets where the model still detects fraud. The mechanism-level fix is channel placement, not a better trade-off on the same channel — moving the noise from the shared weights onto the released contribution signal brings the honest-reward budget from $\epsilon^\star \approx 3000$ down to $\epsilon^\star \approx 50$. Finally, the two defence layers are comple-

mentary rather than redundant: Krum (in its $n \geq 2f+3$ regime) rejects weight-level poisoning that plain averaging cannot survive, while the Shapley-weighted reward economically isolates a colluding majority that out-numbers any statistical defence; and the attribution layer that powers this defence remains tractable at scale only via Monte-Carlo estimation, whose fixed-budget fidelity is itself bounded (top-1-faithful to $n \approx 5$).

5.15.2 Implications

For deployment, the practical consequence is an operating point: a consortium that needs both a useful global model and honest on-chain rewards should leave the weight channel un-noised, enforce privacy on the released contribution channel at $\epsilon \approx 50$, and run Krum sized so that its theorem holds. For research, the chapter's central implication is that privacy and incentive mechanisms in blockchain-FL systems cannot be evaluated in isolation: prior Shapley-on-blockchain work assumes a clean contribution signal, and this chapter shows that assumption fails precisely in the privacy regimes a regulated deployment would require. A high-accuracy global model is not evidence that the reward ledger is fair — the two can and do diverge, which matters all the more because Fabric's ledger records the unfair rewards immutably. Lastly, the negative results carry information of their own: DB-BOA's value is automation and leader selection rather than a detection-accuracy gain over a careful hand-tuned default, and fixed-budget Monte-Carlo Shapley buys feasibility, not free fidelity.

5.15.3 Limitations and Validity

Several scope constraints are stated plainly and revisited in the Conclusion. The leader-selection frequencies and consensus throughput reported by the simulation layer are computed from node resource scores rather than measured; the per-round latency was measured on a single-host, two-organisation Fabric deployment (mean $\approx 2.1\,\text{s}$ over 50 transactions), not on a multi-host production network. They characterise relative behaviour rather than production performance, and blockchain throughput is therefore explicitly outside the "Scalable" claim. The three institutions are stratified volume-splits of a single bank's ULB dataset — a controlled rather than a true cross-institution setting — which is why the deployment Shapley split is near-uniform (Section 5.5) and why the contribution-differentiation results rely on a constructed training-label-noise gradient, disclosed wherever used. The differential-privacy budget $\epsilon = 1.0$ used in the deployed ablation is intentionally tight; the framework's usable-privacy result is the output-channel mechanism of Section 5.9. Finally, the detection figures are reported at the weight-DP-off operating point, with the full DP-on collapse reported transparently in Section 5.4 rather than omitted.

Chapter 6

Conclusion

6.1 Summary

This thesis presented FL-ADTCN, a blockchain-integrated federated learning framework for collaborative financial fraud detection, implemented end-to-end on a real Hyperledger Fabric consortium. The framework resolves the three fundamental barriers to cross-institutional fraud detection. Privacy is protected by federated learning with differentially private weight sharing: raw transaction data never leaves an institution, and shared parameters carry a formal (ϵ, δ) guarantee. Trust is provided by Hyperledger Fabric: consensus decisions and incentive rules are recorded and enforced by the `DBBOAContract` chaincode, which no single organisation controls and the immutable ledger cannot retroactively alter. Incentive alignment is achieved by coupling exact Shapley-value contribution weights to an on-chain token mechanism, so that economic reward tracks measured contribution automatically.

The primary contribution is a *characterisation of the privacy–incentive coupling* of a Shapley-driven, chaincode-enforced reward mechanism on a real permissioned network, together with a concrete fix: differential privacy and contribution-based incentives are in direct tension, and moving the privacy noise from the shared model weights onto the released contribution channel restores rank-faithful, DP-protected on-chain rewards at roughly two orders of magnitude lower privacy budget ($\epsilon^\star \approx 3000 \rightarrow 50$). This central result rests on, but is distinct from, the engineering substrate that makes it possible: the federated aggregation layer (differentially private weight sharing, Krum robust selection, exact/Monte-Carlo Shapley attribution) bound to blockchain-enforced incentives; a hybrid DB-BOA metaheuristic that tunes the ADTCN detector's hyperparameters and selects the consensus leader; and a deliberately minimal linear-Q-learning policy that handles *sequential* leader rotation and, in simulation, adapts to a node whose reliability degrades at runtime – a secondary, consensus-layer component, not a fraud-detection improvement.

Around the central result we contribute a characterisation suite – the decoupling of accuracy from reward fidelity under DP, the economic isolation of a colluding majority, statistical Byzantine fault tolerance, and the cost–fidelity limits of Shapley attribution as the federation grows – that bounds where the mechanism can and cannot be trusted. Rather than claiming algorithmic priority, this work extends prior Shapley-on-blockchain incentive work (FedCoin [17]) with a deployed Hyperledger Fabric implementation and a noise→fidelity→reward-error analysis.

Empirically, the DB-BOA-tuned centralised ADTCN reached 99.85% accuracy and

an MCC of 0.677 on the ULB Credit Card Fraud dataset – a working detector selected automatically without manual search, though it did not exceed a hand-tuned default (MCC 0.785, with the gap and its run-to-run variance reported openly in Chapter 5); DB-BOA's value here is automation and leader selection, not a detection-accuracy gain – and the federated configuration preserved detection quality while keeping each institution's data local. These detection figures are reported at the deployment operating point, in which weight-channel differential privacy is disabled (its $\epsilon = 1.0$ noise near-randomises the global model, as the ablation shows) and privacy is instead enforced on the incentive channel – the very configuration the central contribution shows to be both private and useful. An ablation over FedAvg, FedAvg+Krum, and FedAvg+DP quantified the privacy/robustness trade-offs and makes this cost explicit rather than hiding it. Byzantine resilience was demonstrated on two fronts: at the parameter level, Krum – run in the regime where its theorem holds ($n = 5, f = 1$ and $n = 7, f = 2$) – rejected every one of four weight-poisoning attacks while plain averaging fell to 87.5% balanced accuracy under norm-boosting; at the behavioural level, the Shapley mechanism assigns an always-fraud adversary a near-zero contribution weight, isolating a coordinated majority that out-numbers any statistical defence.

6.2 Concluding Remarks

The broader lesson of this work is that the components of a trustworthy federated system — privacy, robustness, and incentive fairness — cannot be engineered in isolation and simply composed: the privacy layer quietly corrupts the incentive layer long before it visibly damages the model, and only a system built end-to-end, on real ledger infrastructure, makes that interaction measurable. The framework demonstrated here shows that the tension is not fatal — placed on the right channel, a formal privacy guarantee and an honest, immutable reward record can coexist at a practical budget. Within its stated scope, FL-ADTCN offers consortium fraud detection a working template: data that never leaves the institution, decisions that no single organisation controls, and rewards that track measured contribution. The negative results are reported alongside the positive ones in the belief that knowing where a mechanism breaks is as valuable as knowing that it works.

6.3 Limitations and Future Work

The work has clear limitations, stated honestly; each motivates a corresponding direction of future work (Table 4.7 catalogues the implementation-level constraints and Table 4.8 the corresponding roadmap):

- **Data realism.** The three institutions are volume-splits of a single bank's dataset rather than genuinely distinct sources. Future work will extend the consortium beyond three organisations on real multi-bank streams — which would also move the deployment into the $n \geq 2f + 3$ regime where Krum's statistical guarantee holds, rather than the $f = 0$ configuration of the current three-organisation pipeline.

- **Measured infrastructure.** Consensus throughput is simulated, and the per-round latency was measured only on a single-host, two-organisation Fabric deployment (mean $\approx 2.1\,\mathrm{s}$). Future work will deploy and measure a multi-host Fabric network.
- **Privacy budget.** The differential-privacy budget $\epsilon = 1.0$ is deliberately tight and degrades the shared weights. Future work will relax the budget with DP-SGD at the gradient level.
- **Adaptive adversaries.** The demonstrated Byzantine tolerance holds for up to f colluding organisations, and a sufficiently subtle poisoning attack narrows Krum's separation margin. Future work will harden the aggregator against such close-to-honest poisoning.
- **Long-term security.** Beyond addressing the present limitations, future work will investigate quantum-resistant cryptography for the consortium's identity and ledger primitives.

Bibliography

[1] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992. DOI: 10.1007/BF00992698.

[2] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *Theory of Cryptography (TCC)*, 2006, pp. 265–284. DOI: 10.1007/11681878_14.

[3] K. Chaudhuri, C. Monteleoni, and A. D. Sarwate, "Differentially private empirical risk minimization," *Journal of Machine Learning Research*, vol. 12, pp. 1069–1109, 2011.

[4] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[5] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2017, pp. 1273–1282.

[6] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," *arXiv preprint arXiv:1803.01271*, 2018. [Online]. Available: https://arxiv.org/abs/1803.01271.

[7] H. B. McMahan, D. Ramage, K. Talwar, and L. Zhang, "Learning differentially private recurrent language models," in *International Conference on Learning Representations (ICLR)*, 2018. [Online]. Available: https://arxiv.org/abs/1710.06963.

[8] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd. Cambridge, MA: MIT Press, 2018.

[9] ULB Machine Learning Group, *Credit card fraud detection dataset*, Accessed: April 2026, 2018. [Online]. Available: https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud.

[10] S. Arora and S. Singh, "Butterfly optimization algorithm: A novel approach for global optimization," *Soft Computing*, vol. 23, no. 3, pp. 715–734, 2019. DOI: 10.1007/s00500-018-3102-4.

[11] A. Ghorbani and J. Zou, "Data shapley: Equitable valuation of data for machine learning," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 97, 2019, pp. 2242–2251.

[12] G. Wang, C. X. Dang, and Z. Zhou, "Measure contribution of participants in federated learning," in *2019 IEEE International Conference on Big Data (Big Data)*, 2019, pp. 2597–2604. DOI: 10.1109/BigData47090.2019.9006179.

[13] W. Zhang, R.-S. Chen, Y.-C. Chen, S.-Y. Lu, N. Xiong, and C.-M. Chen, "An effective digital system for intelligent financial environments," *IEEE Access*, vol. 7, pp. 155 965–155 976, 2019. DOI: 10.1109/ACCESS.2019.2943907.

[14] Q. Zhuang, Y. Liu, L. Chen, and Z. Ai, "Proof of reputation: A reputation-based consensus protocol for blockchain systems," in *Proceedings of the 2019 International Electronics Communication Conference (IECC)*, 2019, pp. 131–138. DOI: 10.1145/3343147.3343169.

[15] K. Hsieh, A. Phanishayee, O. Mutlu, and P. B. Gibbons, "The non-iid data quagmire of decentralized machine learning," *Proceedings of Machine Learning Research*, vol. 119, pp. 4387–4398, 2020. [Online]. Available: https://arxiv.org/abs/1910.00189.

[16] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proceedings of Machine Learning and Systems (MLSys)*, vol. 2, 2020, pp. 429–450. [Online]. Available: https://arxiv.org/abs/1812.06127.

[17] Y. Liu, Z. Ai, S. Sun, S. Zhang, Z. Liu, and H. Yu, "FedCoin: A peer-to-peer payment system for federated learning," in *Federated Learning – Privacy and Incentive*, ser. Lecture Notes in Computer Science, vol. 12500, 2020, pp. 125–138. DOI: 10.1007/978-3-030-63076-8_8.

[18] H. Al-Shaibani, N. Lasla, and M. Abdallah, "Consortium blockchain-based decentralized stock exchange platform," *IEEE Access*, vol. 8, pp. 123 711–123 725, 2020. DOI: 10.1109/ACCESS.2020.3005663.

[19] K. Tsoulias, G. Palaiokrassas, G. Fragkos, A. Litke, and T. A. Varvarigou, "A graph model based blockchain implementation for increasing performance and security in decentralized ledger systems," *IEEE Access*, vol. 8, pp. 130 952–130 965, 2020. DOI: 10.1109/ACCESS.2020.3006383.

[20] M. Tubishat, M. Alswaitti, S. Mirjalili, M. A. Al-Garadi, M. T. Alrashdan, and T. A. Rana, "Dynamic butterfly optimization algorithm for feature selection," *IEEE Access*, vol. 8, pp. 194 303–194 314, 2020. DOI: 10.1109/ACCESS.2020.3033757.

[21] G. Andrew, O. Thakkar, H. B. McMahan, and S. Ramaswamy, "Differentially private learning with adaptive clipping," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 17 455–17 466.

[22] Y. Fraboni, R. Vidal, and M. Lorenzi, "Free-rider attacks on model aggregation in federated learning," in *Proceedings of the 24th International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. PMLR, vol. 130, 2021, pp. 1846–1854. [Online]. Available: https://arxiv.org/abs/2006.11901.

[23] Y. Liu, X. Ao, Z. Qin, *et al.*, "Pick and choose: A gnn-based imbalanced learning approach for fraud detection," in *Proceedings of the Web Conference 2021 (WWW)*, 2021, pp. 3168–3177. DOI: 10.1145/3442381.3449989.

[24] Association of Certified Fraud Examiners, "Occupational fraud 2022: A report to the nations," Association of Certified Fraud Examiners, 2022, Accessed: April 2026. [Online]. Available: https://www.acfe.com/-/media/files/acfe/pdfs/rttn/2022/2022-report-to-the-nations.pdf.

[25] A. Bicaku, M. Zsilak, P. Theiler, M. Tauber, and J. Delsing, "Security standard compliance verification in system of systems," *IEEE Systems Journal*, vol. 16, no. 2, pp. 2195–2205, 2022. DOI: 10.1109/JSYST.2021.3064196.

[26] Y. Li, X. Wei, Y. Li, Z. Dong, and M. Shahidehpour, "Detection of false data injection attacks in smart grid: A secure federated deep learning approach," *IEEE Transactions on Smart Grid*, vol. 13, no. 6, pp. 4862–4872, 2022. DOI: 10.1109/TSG.2022.3204796.

[27] R. Nourmohammadi and K. Zhang, "An on-chain governance model based on particle swarm optimization for reducing blockchain forks," *IEEE Access*, vol. 10, pp. 118 965–118 980, 2022. DOI: 10.1109/ACCESS.2022.3221419.

[28] L. Wang and Y. Wang, "Supply chain financial service management system based on blockchain IoT data sharing and edge computing," *Alexandria Engineering Journal*, vol. 61, no. 1, pp. 147–158, 2022. DOI: 10.1016/j.aej.2021.04.079.

[29] H. Givi and M. Hubálovská, "Billiards optimization algorithm: A new game-based metaheuristic approach," *Computers, Materials & Continua*, vol. 74, no. 3, pp. 5283–5300, 2023. DOI: 10.32604/cmc.2023.034695.

[30] D. Jung, J. Shin, C. Lee, K. Kwon, and J. T. Seo, "Cyber security controls in nuclear power plant by technical assessment methodology," *IEEE Access*, vol. 11, pp. 15 229–15 241, 2023. DOI: 10.1109/ACCESS.2023.3244991.

[31] D. Li, D. Han, N. Crespi, R. Minerva, and K.-C. Li, "A blockchain-based secure storage and access control scheme for supply chain finance," *The Journal of Supercomputing*, vol. 79, no. 1, pp. 109–138, 2023. DOI: 10.1007/s11227-022-04655-5.

[32] M. K. Hussin Chowdhury, S. Muhammad Imtiyaj Uddin, J. Moon-il, and H. C. Kim, "Integrating federated machine learning in blockchain crowdfunding ecosystem for increased security and transparency," in *2024 30th International Conference on Mechatronics and Machine Vision in Practice (M2VIP)*, 2024, pp. 1–3. DOI: 10.1109/M2VIP62491.2024.10746050.

[33] G. Machhale, P. B.R., and C. N. Modi, "Enhancing data security and privacy through blockchain and machine learning," in *2024 MIT Art, Design and Technology School of Computing International Conference (MITADTSoCiCon)*, 2024, pp. 1–10. DOI: 10.1109/MITADTSoCiCon60330.2024.10575309.

[34] D. Saveetha, G. Maragatham, V. Ponnusamy, and N. Zdravković, "An integrated federated machine learning and blockchain framework with optimal miner selection for reliable ddos attack detection," *IEEE Access*, vol. 12, pp. 127 903–127 915, 2024. DOI: 10.1109/ACCESS.2024.3413076.

[35] V. T. Truong, H. D. Le, and L. B. Le, "Trust-free blockchain framework for ai-generated content trading and management in metaverse," *IEEE Access*, vol. 12, pp. 41 815–41 828, 2024. DOI: 10.1109/ACCESS.2024.3376509.

[36] Q. Yang, W. Xu, T. Wang, *et al.*, "Blockchain-based decentralized federated learning with on-chain model aggregation and incentive mechanism for industrial iot," *IEEE Open Journal of the Communications Society*, vol. 5, pp. 6420–6429, 2024. DOI: 10.1109/OJCOMS.2024.3471621.

[37] Y. Zhao, Y. Qu, Y. Xiang, F. Chen, and L. Gao, "Long-term proof-of-contribution: An incentivized consensus algorithm for blockchain-enabled federated learning," *IEEE Transactions on Services Computing*, vol. 17, no. 5, pp. 2558–2570, 2024. DOI: 10.1109/TSC.2024.3399653.

[38] D. Commey, R. A. Sarpong, and G. V. Crosby, "A bayesian incentive mechanism for poison-resilient federated learning," *arXiv preprint arXiv:2507.12439*, 2025. [Online]. Available: https://arxiv.org/abs/2507.12439.

[39] Y. Li, J. Gui, and Y. Wu, "Towards explainable privacy preservation in federated learning via shapley value-guided noise injection," *arXiv preprint arXiv:2503.12958*, 2025. [Online]. Available: https://arxiv.org/abs/2503.12958.

[40] P. S. C. and M. S. Thanabal, "Advanced financial security system using smart contract in private ethereum consortium blockchain with hybrid optimization strategy," *Scientific Reports*, vol. 15, no. 1, p. 6764, 2025. DOI: 10.1038/s41598-025-89404-3.

[41] C. Ying, F. Xia, D. S. L. Wei, *et al.*, "Bit-fl: Blockchain-enabled incentivized and secure federated learning framework," *IEEE Transactions on Mobile Computing*, vol. 24, no. 2, pp. 1212–1229, 2025. DOI: 10.1109/TMC.2024.3477616.

[42] D. Jaramillo-Velez, G. Biczok, A. Graell i Amat, J. Ostman, and B. Pejo, "Private and robust contribution evaluation in federated learning," *arXiv preprint arXiv:2602.21721*, 2026. [Online]. Available: https://arxiv.org/abs/2602.21721.

[43] M. Zhao, C. Dai, F. Chen, X. Chen, K. Ota, and M. Dong, "SI-ChainFL: Shapley-incentivized secure federated learning for high-speed rail data sharing," *arXiv preprint arXiv:2603.07992*, 2026. [Online]. Available: https://arxiv.org/abs/2603.07992.